



Master's thesis

Master's Programme in Computer Science

Measuring Productivity in AI-Assisted Software Development: A Case Study

Argjira Shala

May 21, 2026

FACULTY OF SCIENCE
UNIVERSITY OF HELSINKI

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki, Finland

Email address: info@cs.helsinki.fi

URL: <http://www.cs.helsinki.fi/>

Tiedekunta — Fakultet — Faculty		Koulutusohjelma — Utbildningsprogram — Study programme	
Faculty of Science		Master's Programme in Computer Science	
Tekijä — Författare — Author			
Argjira Shala			
Työn nimi — Arbetets titel — Title			
Measuring Productivity in AI-Assisted Software Development: A Case Study			
Ohjaajat — Handledare — Supervisors			
D.Sc. Mikko Raatikainen, Ph.D. Nea Pirttinen			
Työn laji — Arbetets art — Level		Aika — Datum — Month and year	
Master's thesis		May 21, 2026	
		Sivumäärä — Sidoantal — Number of pages	
		63 pages, 26 appendix pages	
Tiivistelmä — Referat — Abstract			
AI tools are becoming part of everyday software development, supporting coding, testing, and understanding existing code. However, their overall impact on productivity remains unclear. Existing productivity measures often focus on single aspects, such as coding speed, and fail to capture the full development process. This thesis examined how productivity should be measured in the context of AI-assisted software development. The study introduced the AI Productivity Triangle, which conceptualizes productivity as a balance between efficiency, effectiveness, and sustainability. The study then developed a measurement framework, which was informed by a systematic literature review and refined and applied in a case study conducted at a software company. The case study applied the measurement framework using indicators related to development flow, verification effort, defect signals, and developer experience. The analysis combined repository data from GitHub and Jira with developer survey responses. The results indicate that AI tools support faster code production. At the same time, review and verification remained essential parts of the development process. Many pull requests required changes after review began, and review stages often took more time than coding stages. These patterns appeared across different projects and task types. The findings suggest that productivity in AI-assisted development should be measured using multiple dimensions. Organizations should not focus only on coding speed, but also consider review effectiveness and developer experience. The proposed framework was implemented and evaluated in practice, demonstrating how productivity can be measured in real development environments. ACM Computing Classification System (CCS) Software and its engineering → Software creation and management → Software development process management			
Avainsanat — Nyckelord — Keywords			
AI-assisted software development, software engineering productivity, generative AI			
Säilytyspaikka — Förvaringsställe — Where deposited			
Helsinki University Library			
Muita tietoja — övriga uppgifter — Additional information			
Software study track			

Contents

1	Introduction	1
2	Conceptual Foundations	3
2.1	Perspectives on Productivity Measurement in Software Engineering	3
2.2	Framework Baselines	4
2.3	The AI Productivity Triangle	5
3	Research Design and Methodology	7
3.1	Research Approach	7
3.2	Ethical Considerations	8
4	Systematic Literature Review	9
4.1	Literature Review Method	9
4.1.1	Objective	9
4.1.2	Data Sources	10
4.1.3	Search Strategy	10
4.1.4	Inclusion and Exclusion Criteria	10
4.1.5	Selection Procedure	11
4.1.6	Analysis and Synthesis	12
4.2	Overview of Included Studies	13
4.3	Current State of the Evidence	14
4.4	Efficiency	14
4.4.1	AI Tools Improve Speed and Throughput in Suitable Contexts	15
4.4.2	Task Complexity and Workflow Influence Efficiency Gains	15
4.4.3	Faster Understanding and Onboarding	16
4.4.4	Implications for Measuring Efficiency	17
4.5	Effectiveness	17
4.5.1	Functional Correctness Holds for Short, Isolated Tasks	18
4.5.2	Hidden Quality Risks	18

4.5.3	Human Oversight and Development Practices Remain Essential	19
4.5.4	Weak Integration Practices May Reduce Long-Term Stability	20
4.5.5	Implications for Measuring Effectiveness	20
4.6	Sustainability	21
4.6.1	Higher Throughput Increases Coordination Work	21
4.6.2	Sustainable Adoption Factors	21
4.6.3	Human-AI Collaboration	22
4.6.4	Developer Experience Frameworks Emphasize Sustainability	23
4.6.5	Implications for Measuring Sustainability	23
4.7	Measurement Perspective	24
4.8	Gaps in Current Evidence	24
4.8.1	Lack of Multi-Layer Measurement	24
4.8.2	Non-Functional Quality Gaps	25
4.8.3	Verification Work Remains Unmeasured	26
4.8.4	Limited Long-Term Evidence on Team Dynamics	26
4.8.5	Adoption and Governance are Rarely Measured	26
4.8.6	Limited Experimental Control in Field Studies	27
4.9	From Literature Findings to Measurement Perspectives	27
4.10	Summary of Findings Across Dimensions	27
5	Measurement Framework and Case Study Design	29
5.1	Case Setting and Scope	29
5.2	Metric Selection and Refinement Process	30
5.3	Data Collection and Instrumentation	31
5.4	KPI Framework and the AI Productivity Triangle	31
5.4.1	KPI 1 - AI Usage Level	32
5.4.2	KPI 2 - Lead Time and PR Flow	32
5.4.3	KPI 3 - Verification Burden	33
5.4.4	KPI 4 - Post-Merge Defect Escape (14-Day Window)	34
5.4.5	KPI 5 - SPACE Pulse	35
5.5	Limitations and Mitigations	36
6	Results	38
6.1	Phase 1 Results	38
6.2	Phase 2 Results	42

6.3	Cross-Phase Comparison	46
7	Discussion	48
7.1	Overview of Findings	48
7.2	RQ1: How is Productivity Measured in Software Engineering?	48
7.3	RQ2: Limitations of Existing Metrics	49
7.4	RQ3: Role of Existing Frameworks	49
7.5	RQ4: Operationalization in Practice	50
7.6	RQ5: Insights from Applying Indicators	50
7.7	Comparison with Related Work	52
7.8	Implications for Productivity Measurement	53
7.9	Future Directions for AI-Assisted Software Development	54
7.10	Validity and Reliability	55
8	Conclusions	57
	Bibliography	59
A	Included Sources	
B	Evidence-to-Metrics Mapping	
B.1	Efficiency Metrics Mapping (Throughput, Speed, Task Fit)	i
B.2	Effectiveness Metrics Mapping (Quality, Reliability, Rework)	ii
B.3	Sustainability Metrics Mapping (Human Factors, Coordination, Adoption)	ii
B.4	Integrated Evidence Map (Cross-Dimension Summary)	iii
C	AI-Assisted Software Development Experience Survey Instrument	
C.1	Background and Context	i
C.2	Frequency of AI Assistance by Task Type (Past 2 Weeks)	ii
C.3	Experience Constructs (7-Point Likert Scale)	ii
C.4	Optional Open-Ended Reflection	iv
C.5	Construct Mapping to Sustainability Dimensions	iv
D	Phase 1 Detailed Results	
D.1	Observation Context	i
D.2	Dataset Overview	i
D.3	Project Context	i
D.4	Detailed KPI Results	ii

D.4.1	KPI 1 - AI Usage	ii
D.4.2	KPI 2 - Lead Time and PR Flow	ii
D.4.3	KPI 3 - Verification Burden	iii
D.4.4	KPI 4 - Defect Escape	iii
D.4.5	KPI 5 - Developer Survey	iii
D.5	Summary of Phase 1 Detailed Results	v

E Phase 2 Detailed Results

E.1	Observation Context	i
E.2	Dataset Overview	i
E.3	Project Context	i
E.4	Detailed KPI Results	i
E.4.1	KPI 1 - AI Usage	i
E.4.2	KPI 2 - Lead Time and PR Flow	ii
E.4.3	KPI 3 - Verification Burden	ii
E.4.4	KPI 4 - Defect Escape	ii
E.4.5	KPI 5 - Developer Survey	iii
E.5	Summary of Phase 2 Detailed Results	iv

1 Introduction

Artificial intelligence (AI) tools for software development, including generative AI coding assistants, are changing how software is designed, implemented, and maintained. These tools are increasingly used to support productivity in everyday development work. They can generate code, explain unfamiliar APIs, and suggest corrections in real time. This support can speed up tasks such as coding, testing, writing documentation, and technical investigation activities, including debugging and understanding existing systems. As AI-assisted development becomes more common in practice, organizations also need ways to understand whether these tools actually improve software development productivity.

Existing approaches to measuring productivity in software engineering have evolved over time. Early measures focused on output, such as lines of code or function points [1]. More recent approaches consider delivery performance and developer experience through models such as DevOps Research and Assessment (DORA) [2], Satisfaction-Performance-Activity-Communication-Efficiency (SPACE) [3], and Developer Experience Core 4 (DX Core 4) [4]. These approaches provide useful indicators but were developed before the wide adoption of AI-assisted development.

Faster development with AI tools introduces a new challenge: speed does not necessarily lead to better software. Many teams report shorter development cycles and increased output [5]. At the same time, concerns remain about maintainability, correctness, security, and the amount of rework required later [6]. Developers also take on new responsibilities, including verifying AI-generated outputs, adjusting workflows, and coordinating work with automated tools [7]. As a result, existing productivity measures do not fully capture the effects of AI-assisted development. Many metrics still focus on speed or throughput. However, overall productivity may not improve if the additional effort required for verifying AI-generated outputs, correcting mistakes, and performing rework is taken into account. For this reason, productivity needs a broader definition that considers both technical outcomes and human factors in AI-assisted development.

Based on this motivation, this thesis investigates the following research problem:

How should productivity be measured in AI-assisted software development, balancing efficiency, effectiveness, and sustainability?

The study addresses this problem through the following research questions:

RQ1. How is productivity measured in software engineering?

RQ2. What limitations do existing productivity metrics have in AI-assisted development?

RQ3. How can existing frameworks support productivity measurement in AI-assisted workflows?

RQ4. How can productivity indicators be operationalized in a real software development environment?

RQ5. What insights emerge from applying these indicators in practice?

To answer these questions, the research follows a design science approach [8, 9] that combines conceptual development, literature synthesis, and empirical evaluation in a real organizational setting. The thesis first establishes a conceptual model for productivity in AI-assisted software development through the AI Productivity Triangle. A systematic literature review is then conducted to examine current measurement approaches and to analyze how efficiency, effectiveness, and sustainability appear in research on AI-assisted development. Based on the insights from the literature review, a productivity measurement framework is designed and subsequently evaluated through an industrial case study. Finally, the thesis provides practical recommendations for organizations seeking to measure productivity without degrading software quality or developer well-being.

The thesis is organized as follows. Chapter 2 presents the conceptual foundations of the study, including software engineering productivity measurements, existing framework baselines, and the AI Productivity Triangle as the conceptual model. Chapter 3 describes the research design, methodology, and ethical considerations. Chapter 4 presents the literature review process, synthesizes prior evidence, and identifies measurement gaps related to AI-assisted software development. Chapter 5 introduces the measurement framework and case study design, including the organizational setting, selected indicators, data collection, and study limitations. Chapter 6 reports the empirical findings from the two measurement phases and the cross-phase comparison. Chapter 7 discusses the findings in relation to the research questions and prior literature, and examines practical implications, validity, and reliability. Chapter 8 concludes the thesis by summarizing the main contributions and outlining directions for future research.

2 Conceptual Foundations

This chapter presents the conceptual foundations of the study. It reviews software engineering productivity measurement, discusses existing measurement framework baselines, and introduces the AI Productivity Triangle as the conceptual model guiding the study.

2.1 Perspectives on Productivity Measurement in Software Engineering

The concept of productivity in software engineering has changed over time. Early approaches focused on output [1]. Common measures included lines of code (LOC) and function points. Such measures assumed that more code meant more value. This idea was later criticized. More code does not always mean better software. These metrics often reward quantity instead of quality and ignore the cognitive and exploratory nature of programming work.

Agile practices introduced new ways to measure work [10]. Teams began to use indicators such as velocity and story points. Such indicators help teams plan iterations. However, these indicators do not work well for comparing different teams. Teams also sometimes misuse them as performance metrics instead of planning tools.

DevOps introduced another shift focused on delivery performance and system reliability [11]. The DevOps perspective argues that speed matters only when systems remain stable and recover quickly after failures. This perspective later became formalized through the DORA framework [2].

Despite these changes, a basic problem remains. Software does not have a clear unit of output. Development tasks differ from each other. Activity does not always result in value.

In this thesis, productivity refers to the sustained ability of a software team to turn ideas into reliable, maintainable, and valuable software at an appropriate pace, without degrading human well-being.

For measuring productivity, it is important to clarify the terminology used throughout this

thesis. The terms metric, indicator, and KPI are related but distinct concepts. A metric refers to a measurable quantity derived from data, such as the time between events, the number of commits, or the number of review comments. An indicator refers to a metric used to represent a specific aspect of productivity. For example, review time can be used as an indicator of verification effort. A KPI (Key Performance Indicator) refers to a selected productivity dimension represented by one or more related indicators considered important for the analysis. In this thesis, KPIs represent the main dimensions of productivity: efficiency, effectiveness, and sustainability. The term measurement refers to the process of collecting and calculating these metrics from data sources such as GitHub, Jira, and surveys.

2.2 Measurement Framework Baselines: DORA, SPACE, and DX Core 4

This thesis uses several existing measurement frameworks to interpret productivity signals. The DORA framework [2], originally developed through the DevOps Research and Assessment (DORA) program and later adopted by Google Cloud, measures delivery performance. DORA uses four metrics: deployment frequency, lead time for changes, change failure rate, and mean time to recovery (MTTR). The SPACE framework [3], proposed by researchers from Microsoft, GitHub, and the University of Victoria, expands the concept of productivity. SPACE includes five areas: satisfaction, performance, activity, communication and collaboration, and efficiency and flow. The DX Core 4 framework [4], developed within the Developer Experience (DevEx) research community, focuses on the developer perspective, highlighting feedback loops, cognitive load, flow state, and team quality.

These frameworks are widely used and supported by research [12, 13, 14]. However, these frameworks were created before the wide adoption of AI code generation tools. As a result, these frameworks do not fully measure AI-assisted software development-related factors. Such factors include the amount of AI-generated code, the time needed to verify AI suggestions, and uncertainty about the origin of generated code. In addition, these frameworks do not capture differences between teams that use AI tools at different levels. For this reason, this thesis does not replace these frameworks but builds an additional measurement layer that considers AI-assisted development.

2.3 The AI Productivity Triangle

The existing productivity metrics have conceptual and methodological limits. Many approaches focus mainly on efficiency or output and do not capture quality or human factors. To address these limitations, this thesis introduces the AI Productivity Triangle as a conceptual model for understanding productivity in AI-assisted software development. The model, as illustrated in Figure 2.1, defines productivity as a balance between three dimensions: efficiency, effectiveness, and sustainability. These dimensions together form net productivity, which represents a balanced state where all three are maintained.

Efficiency. Efficiency refers to the amount of resources used to produce a given output, often expressed in terms of time, effort, or throughput [15]. In this thesis, efficiency focuses on speed and throughput of development work and describes how quickly teams produce working changes when developers use AI tools. Typical indicators include coding time, pull request cycle time, lead time for changes, and throughput. Efficiency gains are often easy to observe in AI-assisted workflows. Developers can produce code faster with AI tools. However, speed alone does not mean improved overall productivity.

Effectiveness. Effectiveness refers to the degree to which software meets its intended requirements and quality objectives [15]. In this thesis, effectiveness includes correctness, maintainability, and overall software quality. Indicators include defect rates, verification effort, rework, technical debt, and stability after deployment. Effectiveness is important in AI-assisted development. Faster code generation may increase the need for review, testing, and introduce defects, or create new maintenance work later

Sustainability. In this thesis, sustainability refers to the ability of a software development process to maintain developer performance and well-being over time. This dimension focuses on how AI tools affect developer experience and long-term capability development. Relevant aspects include trust in AI tools, cognitive load, flow, motivation, autonomy, and opportunities to learn.

Net productivity depends on maintaining balance across all three dimensions. For example:

- Faster coding may reduce effectiveness if it requires more verification.
- Higher throughput may reduce sustainability if cognitive load increases.
- Strict quality checks may slow development, but support long-term sustainability.

Productivity in AI-assisted development should also be considered from a value perspec-



Figure 2.1: The AI Productivity Triangle: A multi-dimensional model of AI-assisted software development productivity developed in this thesis.

tive. Faster code generation and higher output do not necessarily produce more valuable software. AI tools may accelerate implementation while still increasing verification effort, maintenance needs, or usability problems. For this reason, productivity should not be evaluated solely through speed or throughput, but through the balance between efficiency, effectiveness, and sustainability.

The AI Productivity Triangle guides the structure of this thesis. The model supports the systematic literature review presented in Chapter 4, informs the design of the measurement framework described in Chapter 5, and guides the interpretation of empirical findings in Chapter 6. More broadly, the model helps evaluate AI-assisted development as a socio-technical system rather than only as a technical acceleration tool.

3 Research Design and Methodology

3.1 Research Approach

This thesis follows the Design Science Research (DSR) approach [8, 9]. Design Science Research studies aim to study, develop, and evaluate artifacts that address practical problems. In software engineering, such artifacts can include models, frameworks, or measurement systems. In this study, the artifact is a productivity measurement framework for AI-assisted software development. The framework builds on the AI Productivity Triangle and uses a set of practical KPIs designed for real development environments.

The study consisted of three connected components. The first component was a systematic literature review. The literature review examined how productivity in AI-assisted software development has been conceptualized and measured in prior research. The literature review also identified gaps in current measurement approaches. These findings formed the knowledge base for the study.

The second component was the constructive design of the measurement framework. This part translated the literature findings into a practical set of KPIs. The constructive part defined what is measured and how the measures are operationalized in practice.

The third component was an industry case study conducted with Basware. The case study collected and analyzed two types of empirical data: repository telemetry data from GitHub and Jira, and developer perception data collected through a survey aligned with the SPACE framework.

Together, these components follow the main stages of Design Science Research as summarized in Figure 3.1. The study started from a practical problem: current productivity measures do not capture AI-assisted development well. This led to the development of the AI Productivity Triangle as a conceptual model for the measurement framework. Prior research was then used as a knowledge base, after which the measurement framework was designed. Finally, the framework was evaluated through the industry case study.

The systematic literature review method and findings are presented in Chapter 4. The constructive design of the KPI framework and the case study setup are presented in Chapter 5. The empirical findings from the case study evaluation are presented in Chapter 6.

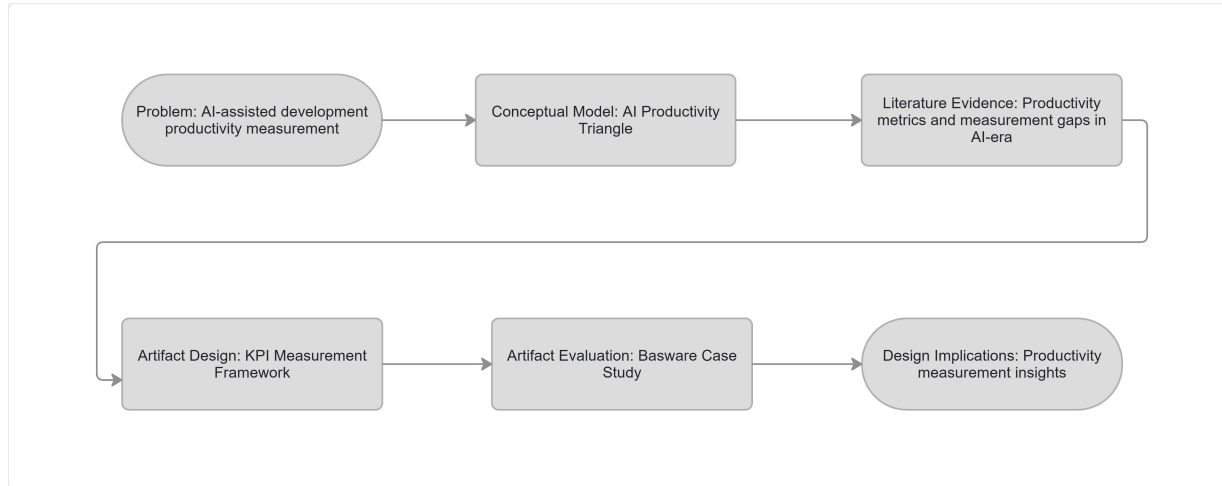


Figure 3.1: Design Science Research process used in this thesis, adapted from [8].

3.2 Ethical Considerations

This study used data from a real software development environment and included a developer survey. For this reason, the study considered basic ethical aspects during the research process. The analysis used aggregated survey results, and the study did not collect personal information or identify individual developers in the analysis or reporting. Repository and issue-tracking data from GitHub and Jira were used only for research purposes in this case study. The analysis focused on development patterns rather than individual performance.

4 Systematic Literature Review

This chapter presents the method and findings of the literature review. The review examines how prior research explores and measures productivity in AI-assisted software development.

The findings are organized according to the three dimensions of the AI Productivity Triangle:

- Efficiency (speed, throughput, flow)
- Effectiveness (quality, correctness, maintainability)
- Sustainability (developer experience, trust, cognitive load, adoption, and coordination)

Where appropriate, the reviewed metrics are related to existing frameworks such as DORA, SPACE, and DX Core 4. The chapter also highlights limitations and gaps in current measurement approaches, which motivated the measurement framework proposed later in this thesis.

4.1 Literature Review Method

4.1.1 Objective

The literature review followed principles commonly used in systematic literature reviews in software engineering [16]. The review aimed to identify definitions, frameworks, and metrics used to describe software engineering productivity; examine how the literature addresses the dimensions of efficiency, effectiveness, and sustainability; analyze conceptual and empirical research on AI-assisted development practices; and identify research gaps that motivate new conceptual approaches.

4.1.2 Data Sources

The study searched three scientific databases to ensure broad coverage: IEEE Xplore, ACM Digital Library, and SpringerLink. Since AI tools evolve quickly, the review also included preprint repositories, namely arXiv and SSRN.

The study also considered selected grey literature. These sources included engineering blogs and technical reports from GitHub, Microsoft, Google, Meta, Medium, and ThoughtWorks. Grey literature was identified through a targeted manual search using the same key terms as in the database search described below. The grey literature provides practitioner perspectives and discussions about new development practices. Several of these ideas do not yet appear in academic publications.

4.1.3 Search Strategy

The search strings were developed through several iterations. The final queries were organized into three groups: productivity terms, metrics and framework terms, and AI-assisted development terms.

Productivity terms were: “software productivity” OR “developer productivity” OR “engineering productivity.”

Metrics and framework terms were: “metrics” OR “measurement” OR “DORA” OR “SPACE” OR “developer experience” OR “DX.”

AI-assisted development terms were: “AI-assisted” OR “large language model” OR “AI-tools” OR “LLM” OR “code generation” OR “AI coding” OR “GitHub Copilot” OR “ChatGPT.”

4.1.4 Inclusion and Exclusion Criteria

The review applied predefined inclusion and exclusion criteria during screening. A study was included if it met at least one inclusion criterion and did not meet any exclusion criteria.

Inclusion Criteria

- Studies that define or discuss software engineering or developer productivity in AI-assisted or AI-augmented development contexts.

- Studies that propose, evaluate, or apply productivity-related metrics or measurement frameworks in AI-assisted development.
- Studies that analyze developer performance, developer experience, or workflow outcomes in AI-assisted software development.
- Studies that examine AI-assisted or AI-augmented software development practices (e.g., LLM-based coding assistants).
- Studies addressing socio-technical, cognitive, or human factors related to AI-assisted software development tools and practices.

Exclusion Criteria

- Studies not related to software engineering or software development practice.
- Studies that do not address AI-assisted or AI-augmented software development.
- Studies focusing exclusively on machine learning or algorithmic model performance without implications for developer productivity or development workflows.
- Publications that do not report definitions, metrics, empirical results, or methodological approaches related to software engineering productivity in AI-assisted development.
- Duplicate publications or preliminary preprints superseded by later versions.
- Studies relying solely on student participants performing artificial tasks without broader relevance to professional software engineering practice.

4.1.5 Selection Procedure

The selection process followed three steps. First, title screening removed studies that were clearly unrelated to the topic. Second, abstract screening assessed whether each study aligned with the research objectives. Finally, a full-text review applied the defined inclusion and exclusion criteria to determine the final set of studies. The study selection process follows a PRISMA-style structure. Figure 4.1 illustrates and quantifies the identification, screening, and inclusion stages of the literature review.

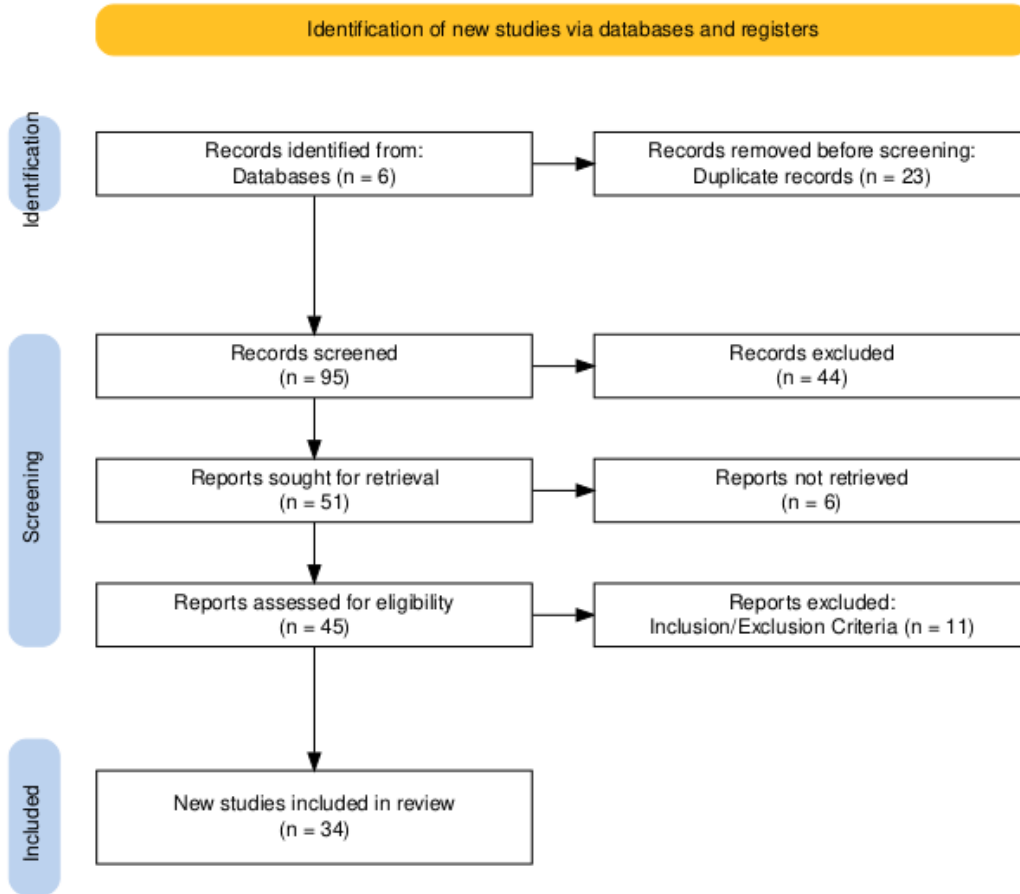


Figure 4.1: PRISMA-style study selection flow for the literature review. Note: In the diagram, “Databases” refers collectively to scientific databases, preprint repositories, and web-based search sources. Generated using the PRISMA 2020 flow diagram tool [17].

4.1.6 Analysis and Synthesis

The study analyzed the extracted data using thematic synthesis. The analysis grouped findings from the reviewed studies into themes based on the three dimensions of the AI Productivity Triangle.

For each study, key information was summarized and stored in a structured database maintained in Notion*. The database included the following columns: title, year, study type, research method, study setting, research questions, relevance to AI-assisted development, productivity dimensions addressed, summary of findings, benefits, challenges, metrics used or discussed, and research gaps. This structure supported consistent comparison across

*<https://www.notion.so/>

studies.

The analysis focused on how productivity is measured in AI-assisted software development. The analysis compared the metrics and indicators used in different studies and examined which aspects of productivity were covered. The analysis also identified differences between studies and related these differences to factors such as task complexity and workflow context. This approach helped identify common patterns as well as gaps in current measurement approaches.

4.2 Overview of Included Studies

The final set of included studies consists of 34 sources. The full list of included sources is provided in Appendix A. All studies were published between 2024 and 2025, reflecting the recent development of AI-assisted coding tools.

The included sources represent several types of evidence, as summarized in Table 4.1. The evidence base includes controlled experiments and randomized evaluations, repository mining studies, enterprise field studies, literature reviews, conceptual measurement frameworks, and practitioner-oriented sources such as industry reports and blogs.

Table 4.1: Distribution of included sources by evidence type

Evidence type	Count
Controlled experiments and randomized evaluations	5
Repository mining and large-scale software analytics	2
Industry and enterprise case studies, field deployments, and empirical reports	8
Systematic literature reviews and evidence reviews	4
Conceptual, framework, and measurement-method papers	5
Practitioner frameworks, industry blogs, and expert commentary	10
Total	34

In terms of focus, efficiency aspects such as speed and throughput were the most frequently examined, while developer experience and sustainability factors received comparatively less empirical attention.

4.3 Current State of the Evidence

Recent research indicates that the field is still developing. A systematic literature review [5] analyzed 39 studies published between 2014 and 2024 and found strong variation in methods and results. Many empirical studies cover short time periods. Many rely on self-reported data. Most studies also examine only a few dimensions of the SPACE framework. This fragmentation is one of the issues addressed in this thesis.

Industry research also proposes new productivity metrics for AI-assisted development. These proposals focus on measures related to time, quality, CI/CD behavior, and cognitive load. The analysis suggests that many of these proposals have not yet been validated in real development environments [18, 19]. Other approaches attempt to improve measurement methods. For example, one study [20] uses COSMIC function points [21] combined with stratified sampling. These approaches aim to measure productivity in a more systematic way. However, the analysis suggests that such approaches may omit factors such as developer experience, rework effort, and CI/CD signals across different development stages.

Together, our findings highlight the need for improved productivity measurement. The insights from this literature review inform the design of the measurement framework proposed in this thesis. The following three sections present the synthesized results according to the dimensions of the AI Productivity Triangle: efficiency, effectiveness, and sustainability. Then the remaining three sections discuss measurement perspectives, research gaps, and implications for framework design.

4.4 Efficiency: Gains Depend on Task and Context

Studies from experiments, open-source projects, and industry deployments indicate that AI coding tools can increase development speed. Developers often complete tasks faster when they use automated assistance. However, the size of these gains varies considerably across contexts. Factors such as task type, programming language, repository complexity, developer experience, workflow integration, and organizational practices all influence the observed productivity effects.

4.4.1 AI Tools Improve Speed and Throughput in Suitable Contexts

AI tools help most with routine and well-defined tasks. Studies [18, 22, 23] report 30-50% faster completion for tasks, such as boilerplate code, CI/CD scripts, documentation updates, and small feature changes. Enterprise telemetry [24] suggests 26-35% reductions in development life-cycle time when developers use AI tools directly inside their normal workflow. A controlled experiment [25] reports similar results. Developers complete about 65% more functional requirements per minute when they use AI tools. They also search the web less often during development.

Consultancy studies indicate similar patterns. A randomized trial in a Finnish software company [26] reported 29% faster task completion when developers used AI tools. When developers also received training on prompting and verification, the improvement reached about 57%. The largest gains appeared in documentation and small improvements. Debugging tasks indicated smaller gains. These results suggest that productivity depends not only on the AI model but also on the type of task and the training developers receive.

In addition to individual task performance, studies [27, 28] report increases in project-level throughput after teams adopt AI tools. An analysis of GitHub projects [27] found that Copilot adoption increased merged pull requests by 5.9%. The rate of timely merges also increased at several time thresholds (+3.5%, +4.1%, +5.1%). Although review effort increased slightly, the increase did not offset the observed throughput gains.

An industry experiment [22] reports similar patterns. Three randomized field studies discussed in the industry experiment found 15-30% increases in pull requests, commits, and builds among developers who used AI tools. Industry reports [18, 29] also describe faster progress across several stages of development, including implementation, testing, and documentation. However, many of these reports rely on observational or practitioner evidence rather than strict experimental control, which limits causal interpretation.

4.4.2 Task Complexity and Workflow Influence Efficiency Gains

Efficiency improvements depend strongly on workflow integration and task complexity. Several studies report that the largest gains occur when AI tools integrate directly into development workflows and environments [23, 30, 31]. An enterprise deployment study [23] followed developers over one year and reported a 33.8% reduction in pull request cycle time

together with a 29.8% reduction in review time. However, the effects on review-related effort appeared to depend heavily on organizational context, task complexity, and review practices. The strongest improvements were observed among junior developers and heavy users of the tools.

Programming language and repository structure also influence outcomes. A study [24] reported the strongest gains in JavaScript, Java, CI/CD-related tasks, and documentation work. In contrast, the weakest improvements appeared in C/C++ projects and complex multi-file systems, where AI tools struggled to maintain architectural understanding across broader contexts.

Workflow design also affects developer experience. Studies [30, 31] report that AI tools integrated directly into development workflows can support focus and developer flow better than tools that require frequent context switching outside the development environment. However, recent agent-based workflows may reduce some of these differences, because developers increasingly interact with AI systems through conversational and multi-file orchestration rather than only through traditional inline code completion inside the IDE.

At the same time, AI tools do not improve efficiency in all situations. A randomized trial [28] examining complex issues in open-source repositories found that experienced maintainers completed tasks approximately 19% slower when using AI tools. Researchers attributed this outcome to limited repository-specific understanding, incomplete or incorrect generated outputs, and additional verification and cleanup effort. These findings suggest that efficiency gains diminish when tasks require extensive contextual understanding, architectural reasoning, or coordination across larger systems.

Training also appears to play a major role in determining productivity outcomes. In the Finnish consultancy experiment [26], training nearly doubled the observed efficiency gains. This suggests that tool access alone is insufficient and that organizations must also support learning, prompting practices, and workflow adaptation [32, 33].

4.4.3 Faster Understanding and Onboarding

Efficiency improvements do not come only from code generation. Enterprise studies [34, 35] report that developers often use AI tools to understand code, search for concepts, or navigate unfamiliar systems. These uses help developers understand new codebases faster and reduce time spent searching for information.

This type of support can reduce onboarding time for new developers and help experienced

developers move faster through unfamiliar parts of the system. Conceptual studies [29, 36] support these findings and warn that organizations must consider training, integration costs, and workflow changes when evaluating productivity improvements.

4.4.4 Implications for Measuring Efficiency

Organizations need appropriate metrics to measure efficiency in AI-assisted development. Simple output measures are often not enough. Metrics should separate different types of tasks and account for AI-generated code. The reviewed studies suggest several useful indicators for measuring efficiency in AI-assisted software development:

- Pull request throughput per developer or team
- Lead time and cycle time, separated into coding, review, and merge stages
- Task categories, such as boilerplate work, multi-file changes, or new logic
- Codebase complexity and maturity
- AI usage indicators, including acceptance rate of suggestions, percentage of AI-generated lines in a pull request, snippet size and frequency, and number of prompt-response cycles
- Onboarding signals, including time to first pull request, number of understanding tasks completed, and reductions in search or context switching

These metrics help organizations separate gross output increases from real efficiency improvements, and also account for review effort, rework, and verification work instead of assuming that higher output automatically means higher productivity.

4.5 Effectiveness: Correct Code Does Not Always Mean Good Software

Studies from experiments, open-source projects, and industry deployments indicate a common pattern. Developers who use AI coding tools often produce solutions that work in the short term. These solutions often pass tests and meet the immediate requirements. However, other quality aspects may suffer. These include performance, maintainability,

and long-term system stability. Without clear review practices and quality checks, these problems may appear later in the development process.

4.5.1 Functional Correctness Holds for Short, Isolated Tasks

Research suggests that AI-assisted solutions often remain correct for small and well-defined tasks. Reported controlled experiments [25] indicate no major drop in correctness for short Python tasks. The maintainability index also remains similar to human-written code. These findings suggest that AI tools can support locally correct solutions when the task scope is limited. The systematic literature review [5], covering 39 studies, reported similar mixed outcomes. Within this review, several studies observed improvements in correctness and maintainability, whereas others identified declines caused by mismatched requirements, generic solutions, or misunderstandings of the broader problem context.

Overall, the findings suggest that effectiveness depends heavily on task complexity, contextual understanding, and the surrounding development process rather than improving uniformly across all situations.

4.5.2 Hidden Quality Risks May Remain Despite Stable Project-Level Metrics

Even when AI-generated code satisfies immediate requirements, additional quality problems may remain hidden. Studies [22, 27, 28, 37] report that approximately 8-25% of AI-generated solutions contain performance-related issues, including slower execution time, higher CPU usage, increased memory consumption, or inefficient resource utilization. These issues are often associated with suboptimal algorithms, inefficient loops, or poor API choices.

Developers can reduce some of these risks through iterative refinement, prompt engineering, and manual optimization. However, these approaches do not eliminate the problem completely. The findings, therefore, emphasize the importance of automated profiling, code review, and architectural oversight in AI-assisted development. Conceptual studies [18, 29] support these observations and argue that quality risks increase when system assumptions remain unclear, undocumented, or poorly communicated across teams.

At the same time, large-scale studies of open-source repositories [27, 28] report that several observable project-level quality indicators remain relatively stable after AI tool adoption.

For example, a study [28] found that the number of issues per pull request and the overall bug-issue ratio remained mostly unchanged despite increased development throughput.

Industry experiments reported in a study [22] describe similarly mixed outcomes. In some organizations, pull requests achieved higher approval rates, whereas in others, build success rates declined slightly. These differences suggest that review culture, testing practices, and organizational workflows strongly influence quality outcomes.

Together, these findings suggest that AI adoption does not automatically lead to immediate quality collapse, but neither does it guarantee improved software quality. Instead, effectiveness depends on how AI-generated outputs are integrated into existing engineering practices.

4.5.3 Human Oversight and Development Practices Remain Essential

Training appears to significantly influence the quality of AI-assisted development outcomes. In a randomized pilot study [26], access to AI tools alone did not reliably improve quality. However, when developers also received structured training, quality scores increased by approximately 45%. The strongest improvements appeared in testing-related tasks, whereas debugging tasks showed smaller changes.

These findings suggest that effectiveness depends not only on the tools themselves, but also on how teams use them. Prompting practices, testing strategies, review processes, and organizational guidance all influence the final quality of AI-assisted outputs.

Developer surveys reported in a study [35] also indicate mixed perceptions of AI-generated code quality. Developers reported modest improvements in code quality when they used AI assistance tools. At the same time, many developers report ongoing challenges.

Common concerns include incorrect suggestions, misleading outputs, and extra time spent on verification and cleanup [6]. Senior developers often report more trust concerns than junior developers. According to a study [31], chat-style AI tools may help developers understand problems or explore solutions. However, these tools do not remove the need for manual review.

4.5.4 Weak Integration Practices May Reduce Long-Term Stability

Industry studies [22, 38] warn about the risk of illusory productivity, where developers appear more productive in the short term while long-term system stability and maintainability deteriorate. Faster implementation alone may therefore create misleading productivity signals if review, testing, and architectural validation do not keep pace.

Important safeguards include smaller pull requests, strong review practices, automated profiling, and comprehensive testing processes. Conceptual studies [18, 29] report similar concerns and argue that teams may trade short-term implementation speed for long-term quality risks when AI-generated outputs are accepted without sufficient validation.

4.5.5 Implications for Measuring Effectiveness

Organizations need metrics that measure more than simple correctness. Effectiveness also includes maintainability and the effort required to prepare code for production. Based on the reviewed literature, useful effectiveness indicators include:

- Review edit distance (difference between AI-generated suggestions and the final merged code)
- First-pass CI or test success rate
- Change failure rate (CFR)
- Post-merge rework within 14 days
- Maintainability indicators such as linter violations, complexity scores, and readability metrics
- Performance profiling for pull requests that include AI-generated code (runtime, CPU, memory usage)
- Bug ticket density linked to AI-generated code
- Verification effort or verification time (when telemetry is available)

These metrics help teams detect issues that simple tests may miss. These metrics also help organizations understand whether AI-assisted development improves real software quality or only speeds up initial code creation.

4.6 Sustainability: Human and Organizational Factors

Efficiency and effectiveness are important dimensions of productivity. However, long-term productivity also depends on human and organizational factors. These include developer experience, coordination within teams, adoption patterns, and governance practices. Studies from open-source projects, industry deployments, and mixed-method research highlight several sustainability themes in AI-assisted development.

4.6.1 Higher Throughput Increases Coordination Work

When AI tools speed up coding tasks, other activities may take more time. Studies of open-source projects [27, 28] report that faster coding can increase coordination work. Merge times rise by about 8%, pull requests involve more reviewers, and discussions during reviews become longer.

These findings suggest that faster implementation shifts work toward review and coordination. Productivity remains positive when teams have enough capacity to handle this extra work.

4.6.2 Sustainable Adoption Depends on Organizational and Workflow Factors

Studies [22, 37, 39] report that integration with existing workflows strongly influences AI tool adoption. A study [30] found that compatibility with current tools and practices predicts adoption more strongly than perceived usefulness or expected productivity benefits. Enterprise deployments [6, 23, 31, 35] report similar observations. Adoption rates are generally higher when AI tools support existing technology stacks, CI workflows, and established development practices. These findings suggest that sustainable productivity depends less on the AI model itself and more on how well the tools fit everyday development work.

Studies [6, 22, 23] also report differences in how developers adopt AI tools. Junior developers often describe faster onboarding, easier implementation, and fewer context switches when using AI tools. Senior developers appear more cautious and tend to focus more

strongly on verification work, architectural correctness, and maintainability concerns. These differences may create uneven adoption patterns within teams and contribute to differences in trust, review behavior, and workflow practices.

Long-term sustainability also depends on broader organizational conditions. A study [35] reports that sustained AI usage depends on trust, transparency, and opportunities for learning and adaptation. Developers rarely accept AI-generated suggestions without review. Instead, most developers inspect, modify, and contextualize generated outputs before merging them into production systems.

Additional concerns reported in the literature [6, 40] include pressure to adopt AI tools, fairness in developer performance evaluation, and the risk of skill degradation among specialists. An industry report [41] describes additional organizational risks, including unmonitored tool usage (“shadow AI”), prolonged pilot phases, and weak integration with development workflows. These factors may reduce the long-term value and sustainability of AI adoption.

4.6.3 Human-AI Collaboration Influences Learning, Flow, and Cognitive Load

Many developers use AI tools not only for code generation, but also for understanding systems and supporting learning activities. Studies [34, 35] report that developers frequently use AI assistance to read unfamiliar code, understand concepts, explore examples, and navigate large codebases. These activities support knowledge development and may reduce onboarding time in complex systems. The benefits appear strongest when developers combine AI assistance with code review, team collaboration, and existing engineering practices rather than relying solely on automated outputs.

Research also reports mixed effects on cognitive load and developer flow. Studies [29, 42] report reduced cognitive load because AI tools provide explanations and reduce time spent searching for information. Other studies [5, 19, 31] report increased cognitive load caused by repeated prompting, verification effort, and the need to evaluate generated outputs critically.

Flow appears to improve when AI tools integrate naturally into development workflows and environments. In contrast, flow may decline when tools interrupt work patterns or require frequent switching between interfaces. Research [29] also suggests that training and gradual workflow adaptation help teams maintain productive and sustainable work

practices.

The quality of interaction between developers and AI tools also influences sustainability outcomes. A conceptual framework [19] proposes evaluating collaboration through indicators such as edit distance between generated and final code, the number of developer-tool interaction cycles, and the depth of review activity. These indicators reflect how actively developers engage with AI-generated outputs rather than accepting them passively.

Studies [7, 26, 35] similarly report that developers usually review and modify generated code instead of accepting it directly. These findings suggest that AI tools currently function more as collaborative assistants than as autonomous software producers.

4.6.4 Developer Experience Frameworks Emphasize Sustainability

Developer experience frameworks, namely SPACE and DX Core 4, highlight several important factors for sustainable productivity. These frameworks include dimensions such as satisfaction and well-being, focus and flow, cognitive load, team quality, feedback loops, and business impact.

As reported in these multiple studies [43, 44, 45, 46, 47], organizations increasingly consider these factors when evaluating AI-assisted development practices. These perspectives help organizations balance faster development with sustainable working conditions.

4.6.5 Implications for Measuring Sustainability

To evaluate sustainable productivity, organizations need indicators that capture human experience and team dynamics. Human and team experience can be observed through developer satisfaction, perceived usefulness of AI tools, focus and flow during work, cognitive load, learning and skill development, and trust in AI suggestions.

Verification work can be reflected in the time required to obtain a useful AI suggestion, the share of AI-generated code that developers revise or discard, and the depth of code review, including comments and edit distance. Coordination and collaboration can be observed through the number of reviewers per pull request, review latency, merge time across development stages, and cross-team interactions.

Equity in adoption can be examined by differences in AI tool usage, trust in AI suggestions,

perceived benefits, and verification effort among developers. Organizational conditions include the presence of clear policies for AI use, quality checks for AI-generated code, and the prevention of untracked tool usage (“shadow AI”).

These indicators help organizations understand how AI tools affect developers, teams, and workflows. Such indicators also support the evaluation of long-term productivity and organizational stability.

4.7 Measurement Perspective: What is Measured Shapes the Result

Studies also differ in how they measure productivity. These differences influence the conclusions they report. Studies that measure gross output, such as coding speed or the number of completed tasks, often report improvements of 30-40%. A study [48] that measured net productivity, where measurements included verification time, review effort, and rework, reported smaller but still positive gains of about 15-20%.

Studies that focus only on coding time, the number of tasks completed, or simple correctness tests often report larger improvements. However, studies that include additional factors, such as rework, review depth, long-term quality, and coordination effort, often report more mixed or moderate results.

These findings indicate an important point: the choice of measurement determines what appears to improve and what remains hidden.

4.8 Gaps in Current Evidence

Research on AI-assisted software development has grown quickly. However, several gaps remain. These gaps explain why studies report different results. They also explain why there is not yet a shared productivity framework for AI-assisted development.

4.8.1 Lack of Multi-Layer Measurement

Most studies examine only one aspect of productivity. Repository or CI-based studies often exclude developer experience, survey-based studies typically lack behavioral data, and code-quality studies frequently overlook human factors.

No existing study combines repository activity, code quality, and developer experience for the same contributions. Because of this limitation, we cannot observe the full system, and it remains difficult to understand how AI tools influence both short-term output and long-term sustainability.

4.8.2 Non-Functional Quality and Long-Term Sustainability Remain Undermeasured

Laboratory experiments indicate that AI-generated code may contribute to performance problems. Studies [22, 27, 28, 37] report approximately 8-25% performance regressions in generated code. These issues include inefficient runtime behavior, increased memory consumption, and poor API usage. However, real-world studies rarely measure such effects directly. Most field studies do not examine runtime behavior, performance anti-patterns, security vulnerabilities, or architectural consistency.

Instead, many studies rely on relatively simple indicators such as test correctness, pull request approval, or build success. These measures provide limited insight into long-term system quality and maintainability. AI-generated code may accelerate implementation while gradually introducing design drift, inconsistencies, or maintenance challenges that become visible only later in the software lifecycle.

The literature also provides limited evidence regarding broader sustainability and organizational concerns. Few studies examine whether AI tools benefit developers equally or whether they amplify differences in experience, expertise, or confidence. Similarly, security-related concerns, such as vulnerability propagation and unsafe coding patterns, are rarely incorporated into productivity measurement. In addition, the long-term effects of AI reliance on developer learning, knowledge retention, and architectural understanding remain unclear.

These gaps limit the ability to evaluate productivity from a broader sustainability perspective. Current productivity metrics appear better suited for capturing short-term efficiency gains than long-term software quality, maintainability, security, and organizational capability development.

4.8.3 Verification Work Remains Unmeasured

Several qualitative studies describe the effort required to review AI-generated code. Developers often spend time validating AI suggestions, rewriting prompts, cleaning up generated code, and debugging AI-generated artifacts. However, few studies measure this work directly. Existing research rarely quantifies verification effort using indicators such as time to the first useful edit, the difference between suggested and final code, cleanup changes in pull requests, short-term rework after merging, or the depth of code review when AI-generated code is involved.

Conceptual studies [19, 39] propose metrics for collaboration quality. Examples include edit distance from generated output and the rate at which developers detect incorrect suggestions. These ideas remain mostly conceptual and rarely appear in empirical studies. Without such measurements, we cannot estimate real productivity effects.

4.8.4 Limited Long-Term Evidence on Team Dynamics

Several studies discuss social and organizational risks of AI adoption. Examples include reduced peer interaction and mentorship, uneven adoption or trust across developers, pressure to use AI tools, differences in confidence between junior and senior developers, and informal or untracked use of AI tools (“shadow AI”). However, no long-term datasets track these effects. Studies have not yet observed how knowledge flows within teams over time, how collaboration patterns change, or how developer skills evolve with AI assistance. Because of this gap, the long-term organizational impact of AI adoption remains unclear.

4.8.5 Adoption and Governance are Rarely Measured

Previous studies reported that workflow compatibility influences tool adoption, which in turn affects productivity. However, field studies rarely measure governance and integration factors. For example, few studies track whether code originates from AI tools, development policies for AI usage, review practices for AI-generated code, or training and organizational readiness for AI adoption. Many discussions about governance remain conceptual and rarely connect governance practices to measurable development outcomes.

4.8.6 Limited Experimental Control in Field Studies

Real-world studies provide useful insights; however, they are often difficult to interpret. Several factors limit causal conclusions, including the fact that developers move between AI-assisted and non-assisted workflows, adoption levels differ across developers and teams, and technology stacks vary between organizations.

Few studies combine experimental variation with detailed development telemetry. Pilot experiments [26, 49] report positive effects but also indicates practical limitations. Examples include participant dropout, reliance on self-reported time estimates, and possible reviewer bias.

These limitations suggest the need for better field experiments that combine controlled interventions with repository telemetry.

4.9 From Literature Findings to Measurement Perspectives

The literature review suggests that productivity in AI-assisted software development should be measured using multiple complementary indicators rather than simple output measures alone. The findings highlight the importance of combining development flow, quality-related signals, developer experience, and AI usage context.

This perspective follows the three dimensions of the AI Productivity Triangle: efficiency, effectiveness, and sustainability. Together, these dimensions support a multi-layer measurement approach that combines repository telemetry, workflow signals, and developer perception data.

The literature findings presented in this chapter inform the KPI framework developed for the case study in Chapter 5. The detailed mapping between literature findings and candidate measurement signals is provided in Appendix B.

4.10 Summary of Findings Across Dimensions

Table 4.2 summarizes the main findings from the literature across the three dimensions of productivity. The table highlights key patterns, influencing factors, and implications for

measurement.

Table 4.2: Summary of productivity dimensions, findings, and measurement implications

Dimension	Main findings	Key factors	Implications for measurement
Efficiency	AI tools increase coding speed, especially for routine tasks. Gains vary across tasks and contexts.	Task type, developer experience, tool integration, programming language, training.	Separate coding, review, and merge stages. Account for task type and AI usage. Avoid using output alone.
Effectiveness	AI-generated code is often functionally correct, but may introduce performance issues, rework, or maintainability problems.	Task complexity, prompting quality, review practices, training, system context.	Measure rework, review effort, defect signals, and maintainability. Go beyond correctness and test success.
Sustainability	AI shifts work toward review, coordination, and cognitive effort. Developer experience and adoption vary.	Workflow integration, team practices, trust, cognitive load, organizational support.	Include developer experience, review workload, coordination effort, and adoption patterns. Consider long-term effects.

5 Measurement Framework and Case Study Design

This chapter presents the measurement framework and the empirical case study design used in this thesis. The chapter describes the organizational setting, the process used to refine the KPI framework, the data collection and filtering procedures, and the operationalization of the selected productivity indicators. The chapter also presents the limitations of the study and the mitigation approaches applied during the analysis.

5.1 Case Setting and Scope

This study examined how productivity indicators can be applied in a real software development environment. The goal of the study was to examine whether AI-assisted development was associated with measurable productivity signals in a real organizational setting. The case study was conducted in collaboration with Basware, a software product company that develops solutions for invoice lifecycle management. The collaboration focused on designing a practical measurement approach and evaluating it in an organizational context. The setup was prepared through a series of meetings between November 2025 and January 2026, during which the measurement approach and KPI set were defined.

A steering group was formed with representatives from Basware. The group consisted of one primary contact who coordinated the study from the company side and two additional participants who joined selected meetings. The group supported the design of the study and helped ensure that the selected indicators were relevant and applicable in practice.

Four projects were selected by Basware as suitable candidates for the analysis. The projects differed in size, task type, and development context. Some projects focused more on bug fixes, while others focused on feature development. The projects also varied in scope and complexity. This variation provided a useful context for observing different development patterns. The selected projects were part of an established product environment. The work was therefore mainly brownfield development, including bug fixes and new features added to existing systems. This context is important because existing processes, shared ownership, and established review practices can increase the need for validation and coor-

dination. For confidentiality reasons, the project names are anonymized in this thesis and reported as Projects A, B, C, and D. The selected projects involved 26 developers during the study period. The developer survey was made available only to developers working in these selected projects.

After the preparation phase, the study continued with two measurement phases. Phase 1 covered the period from February 9, 2026, to March 2, 2026, and provided the main dataset. Phase 2 covered the period from March 3, 2026, to March 20, 2026, and extended the observation period and enabled comparison over time.

The study observed ongoing development activity without introducing controlled experiments or major process changes. Developers in the selected projects were encouraged to use AI tools during the study, but tool use was not mandatory, and no controlled treatment group was introduced. The only process change was the addition of a Jira field for capturing AI tool usage. Data were collected from GitHub, Jira, and a developer survey. Details of data collection are described in Section 5.3.

5.2 Metric Selection and Refinement Process

The KPI framework used in this case study was based on the literature review presented in Chapter 4. The detailed mapping between literature findings and candidate metrics is provided in Appendix B. The literature identified a broad set of possible productivity indicators for AI-assisted software development. These indicators covered efficiency, effectiveness, and sustainability.

This set was refined together with Basware during the preparation phase. The selection followed four practical constraints: feasibility with existing data sources, low-friction integration without intrusive monitoring, suitability for a short study window (i.e., ability to provide directional signals within weeks), and comparability by analyzing results across repositories or projects. The approach also supports later extension to a broader organizational level.

Within this structure, the framework distinguishes between two types of signals:

- Priority KPIs represent indicators that are both supported by the literature and reliably measurable in the organizational context.
- Exploratory/supporting signals provide additional context but are not treated as

primary productivity indicators.

5.3 Data Collection and Instrumentation

Data were collected from three sources: GitHub, Jira, and a developer survey. GitHub data included pull request metadata, timelines, commits, and review activity. Python scripts were used to extract this data through the GitHub API. Jira data included task information and defect signals. Pull requests were linked to Jira issues using a Jira key included in the pull request title. This linkage allowed the study to connect development activity with task context. One additional Jira field was added to capture AI usage. Developers selected one of four levels: none, low, medium, or high.

Data were filtered in two steps. First, pull requests without a valid Jira key were excluded from Jira-linked analysis since they could not be connected to task context or AI usage information. Second, pull requests sharing the same Jira key were excluded from AI-segmented analysis to avoid ambiguous attribution of AI usage to a single pull request. Therefore, the analysis distinguishes between all merged pull requests, Jira-linked pull requests, and pull requests suitable for AI-segmented analysis.

A short developer survey was used to capture developer experience related to AI-assisted development. The survey instrument was developed based on concepts from the SPACE framework and related literature. The initial version of the survey was reviewed together with representatives from Basware and tested using a think-aloud approach with two PhD students. This process helped improve the structure, clarity, and wording of the survey. The final survey included questions on AI usage, developer experience, and perceived effects of AI tools. The survey was conducted at the end of each measurement phase.

5.4 KPI Framework and the AI Productivity Triangle

The case study uses five KPIs. Each KPI reflects one aspect of productivity. The framework does not combine the indicators into a single productivity score. Instead, the framework examines how indicators relate to each other. This approach helps identify trade-offs between coding speed, verification effort, quality, and developer experience. This supports the evaluation of net productivity. The following sections describe each KPI.

5.4.1 KPI 1 - AI Usage Level

KPI 1 measures the level of AI assistance used in each pull request and provides context for interpreting the other indicators. AI productivity effects vary across tasks, developers, and workflows. Without measuring AI usage, changes in speed, verification effort, or defect rates cannot be interpreted reliably.

Operational Definition

Each pull request is linked to a Jira issue using a Jira key. Developers indicate the level of AI assistance in a Jira item by selecting one of the following options:

- None - No AI assistance used
- Low - AI used for minor support (e.g., small suggestions, snippets)
- Medium - AI used for a significant part of the work
- High - AI used for most of the work in this item

Analytical Role

This KPI allows segmentation of all other indicators.

5.4.2 KPI 2 - Lead Time and PR Flow

Dimension: Efficiency

Lead time measures the time required for a change to move from implementation to merge.

Operational Definition

Lead time was measured using GitHub timestamps and divided into two stages:

1. **Coding stage:** time from the first commit on the pull request branch to pull request creation
2. **Review stage:** time from pull request creation to pull request merge

The timestamps for the first commit, pull request creation, and pull request merge were extracted from GitHub metadata. Additional supporting contextual signals include reviewer count, review comment volume, PR size (lines added or removed), and number of commits.

Analytical Role

This KPI indicates whether AI reduces overall delivery time or shifts work from coding to review.

Segmentation by AI usage allows patterns such as:

- high AI usage + shorter coding stage + longer review stage → verification shift
- high AI usage + shorter total lead time → possible net productivity gain

5.4.3 KPI 3 - Verification Burden

Dimension: Effectiveness

Verification burden measures the effort required to review and correct code before merging.

Operational Definition

Verification burden is measured using pull request review signals from GitHub metadata. Since CI data were not consistently available across repositories, the analysis focuses on review-stage indicators.

Signals include:

- Post-review rework incidence: percentage of PRs with commits pushed after the first review activity (first review comment or formal review submission).
- Post-review rework ratio: number of commits pushed after first review activity divided by total commits in the PR.
- Number of review rounds: count of distinct review submissions and/or change-request cycles.
- Review comment density: number of review comments normalized by PR size (e.g., comments per 100 lines changed).

For interpretation, the analysis also segments results by PR size (lines added or removed, categorized into small, medium, and large buckets), reviewer count, and Jira task type (e.g., Bug, Story, Refactor), where available.

Analytical Role

KPI 3 measures the effort required to prepare code for merging. Combined with KPI 2, it indicates whether AI reduces total effort or shifts work toward review.

5.4.4 KPI 4 - Post-Merge Defect Escape (14-Day Window)

Dimension: Effectiveness

This KPI provides a proxy for whether changes require corrective work shortly after merging.

Operational Definition

A merged PR counts as a defect escape when one of the following occurs within 14 days: a Jira issue of type Bug referencing the same Jira key, a corrective hotfix pull request, or a revert referencing the original PR. A 14-day window was selected as a practical compromise between sensitivity and attribution. It is long enough to capture many immediate post-merge corrective actions, while short enough to reduce the likelihood that later defects are caused by unrelated subsequent changes. The window also supports comparison within the relatively short observation period of this study.

Defect Escape Proxy Rate (14d):

$$\frac{\text{Number of PRs with at least one defect signal within 14 days}}{\text{Total number of merged PRs}}$$

Analytical Role

This KPI approximates the change failure rate (CFR) under study conditions by indicating whether faster development leads to downstream instability.

5.4.5 KPI 5 - SPACE Pulse

Dimension: Sustainability

Repository telemetry does not capture developer experience or workflow conditions. For this reason, the study includes a short survey.

Operational Definition

The sustainability dimension is measured through a lightweight pulse survey conducted twice during the study. The survey has three components.

1. AI Tool Usage Context

Participants report:

- current level in Basware's internal AI training program (none, Silver, or Gold)
- AI tools used (for example, Copilot, and/or Cursor)
- frequency of AI use across tasks such as implementation, debugging, testing, design, documentation, review support, and code understanding

2. Experience Constructs

Participants respond to statements on a 7-point scale from strongly disagree to strongly agree. The constructs measure:

- verification and rework effort
- flow and focus
- learning and understanding
- decision support
- trust in AI suggestions
- engagement and autonomy

3. Optional Open-Ended Reflection

Participants may add short written comments about their experience.

The full survey is provided in Appendix C.

Analytical Role

KPI 5 provides context for the repository indicators and helps explain patterns such as faster delivery combined with higher perceived verification effort, stable defect rates alongside declining trust in AI tools, and high AI usage combined with improved flow and stable rework. KPI 5, therefore, helps interpret whether productivity changes remain sustainable.

5.5 Limitations and Mitigations

This case study has limitations. These limitations result from the use of existing development data, the short observation period, and the use of self-reported signals. To support interpretation, the analysis includes mitigation steps. These steps reduce the impact of the limitations but do not remove them completely. Table 5.1 summarizes the key limitations of the measurement framework and the corresponding mitigation approaches used in the study.

Table 5.1: Limitations of the case study framework and mitigation strategies

Limitation	Mitigation
AI usage is self-reported	Results are interpreted as correlations rather than causal estimates. Segmentation by PR size, task type, and repository reduces confounding effects.
Review practices differ across teams	Analysis is conducted within repositories or teams rather than across teams.
CI pipeline data were not consistently available across repositories	Review-stage indicators from GitHub metadata were used as comparable proxies for verification effort.
Not all defects are linked back to the originating PR	Defect signals are treated as indicative rather than exhaustive.
Survey responses are anonymous and subjective	Survey results are analyzed descriptively and interpreted together with repository telemetry.
Short observation window	Results are interpreted as directional insights rather than long-term conclusions.

6 Results

This chapter presents the results of the case study across the two measurement phases. The findings are organized according to the KPI framework introduced in Chapter 5, covering AI usage, development flow, verification effort, defect escape, and developer experience. Phase 1 and Phase 2 are first presented separately, followed by a cross-phase comparison.

6.1 Phase 1 Results

Dataset Overview

The Phase 1 dataset included 75 merged pull requests, of which 61 were linked to Jira issues and 49 were suitable for AI-segmented analysis. Table 6.1 summarizes the pull request counts across the four projects.

Table 6.1: Pull request counts in Phase 1

Project	Merged PRs	Jira-linked PRs	AI-segmented PRs
A	7	7	6
B	4	2	2
C	31	23	17
D	33	29	24

Detailed project-level KPI tables are provided in Appendix D.

Project Context

The projects differ in task type and size of changes. These differences affect the development flow and verification effort. Table 6.2 presents the distribution of Jira issue types, while Table 6.3 summarizes the pull request size distribution.

Projects A and D were dominated by bug-fix work, while Project C was dominated by feature work. Project B appeared mixed but contained limited data. Smaller pull requests

Table 6.2: Jira issue types in Phase 1

Project	Bug	Story	Other
A	5	0	1
B	1	1	-
C	4	19	-
D	17	10	2

Table 6.3: Pull request size distribution in Phase 1

Project	Small	Medium	Large
A	3	2	1
B	2	0	0
C	18	7	6
D	22	9	2

appeared more often in bug-fix work, while larger pull requests appeared more often in feature work. Because of these differences, the results are interpreted within the project context.

KPI 1: AI Usage

AI usage varies across projects. Medium AI usage is the most common level in Phase 1. High AI usage appears more often in feature-related work, while low or no AI usage appears more often in bug-fix tasks. Table 6.4 summarizes the AI usage distribution.

Table 6.4: AI usage distribution in Phase 1

Project	None	Low	Medium	High	Total
A	2	1	2	1	6
B	0	1	0	1	2
C	2	4	5	6	17
D	2	3	14	5	24

This pattern suggests that developers often use AI tools as partial support rather than

full automation.

KPI 2: Lead Time and PR Flow

Across all projects, review time is longer than coding time. In this study, the coding stage was measured as the time from the first commit on the pull request branch to pull request creation, based on GitHub timestamps. This measure does not include earlier work on the Jira issue before the first recorded commit. Pull requests were often created soon after the first recorded commit, while review stages took more time and dominated the pull request lifecycle.

Table 6.5 illustrates this pattern using Project C, which mainly consists of feature work.

Table 6.5: Development flow in Phase 1 (Project C)

AI Level	Coding (h)	Review (h)	Total (h)
None	0.3	18	19
Low	1.5	29	31
Medium	3.0	35	38
High	6.5	63	69

Higher AI usage often appears together with shorter coding stages. However, longer total time is usually associated with larger changes rather than AI usage itself. In this analysis, task size was approximated using pull request size, measured as the total number of added and deleted lines. Pull requests were further grouped into small, medium, and large size categories. These results suggest that pull request size and task complexity have a strong effect on lead time.

KPI 3: Verification Effort

Many pull requests required changes after the review process had started. Roughly half of the pull requests included post-review commits. Verification effort was present in most cases. AI usage did not remove the need for rework. Pull requests with AI assistance still required review and correction.

Table 6.6 presents the verification results for Project C. The pattern is representative of the broader Phase 1 dataset: rework remains common, and review rounds increase with

larger and more complex changes.

Table 6.6: Verification results in Phase 1 (Project C)

AI	Rework	Rounds
None	~33%	2
Low	~60%	3
Medium	~65%	3
High	~75%	4

Projects with larger pull requests are associated with higher verification effort, while smaller changes are easier to review. The number of review rounds stays fairly stable across different AI usage levels, but increases when task complexity grows.

KPI 4: Defect Escape

Several pull requests showed defect escape signals within the 14-day window. These cases often relate to larger or more complex changes. Table 6.7 summarizes the observed defect signals across the projects.

Table 6.7: Defect signals in Phase 1

Project	Rate	Context
A	~17%	High AI
B	0%	-
C	~22%	Medium / High
D	~10%	Medium

Defect escape appears in both medium and high AI usage categories. However, the pattern depends not only on AI usage level but also on project context and task type.

KPI 5: Developer Experience

The survey received 22 responses. Most developers had not completed formal AI training: 19 participants reported no training, while one participant reported Silver-level training

and two reported Gold-level training. GitHub Copilot was the most commonly used tool, while a few developers also reported using Cursor.

Developers mainly used AI tools for writing code, testing support, and understanding existing code. Usage was lower for design and architecture tasks and for review support. Survey responses suggest that AI tools help developers stay focused and continue work without interruptions, but they also indicate that AI-generated outputs require verification and sometimes rework. Trust in AI tools remains moderate, and developers rely on their own judgment and remain responsible for decisions.

Open-ended responses indicated a similar pattern. Developers described AI tools as useful for generating unit tests, exploring solutions and supporting repetitive tasks. At the same time, they preferred small suggestions rather than large generated outputs. They also reported that AI tools appeared less effective in complex system contexts and could still produce incorrect outputs.

Combined Interpretation

Coding stages were often short, and developers could produce initial code quickly. However, review and verification still took time, and this part of the process did not become shorter. Rework remained common, and developers continued to check and adjust code before merging. Task complexity had a strong effect on the results. Large changes took more time and required more verification.

Overall, the results indicate that work was distributed differently. Initial coding became faster, while verification remained necessary. Effort appeared to shift from coding toward review and validation rather than being eliminated.

6.2 Phase 2 Results

Dataset Overview

The Phase 2 dataset included 45 merged pull requests, of which 35 were linked to Jira issues, and 31 were suitable for AI-segmented analysis. Table 6.8 summarizes the pull request counts across the projects.

Detailed project-level KPI tables are provided in Appendix E.

Table 6.8: Pull request counts in Phase 2

Project	Merged PRs	Jira-linked PRs	AI-segmented PRs
A	9	8	7
B	5	4	4
C	26	20	18
D	5	3	2

Project Context

The same four projects were analyzed in Phase 2. Task profiles remain different across projects, and pull request size still varies. Table 6.9 presents the Jira issue types, while Table 6.10 summarizes the pull request size distribution.

Table 6.9: Jira issue types in Phase 2

Project	Bug	Story	Other
A	3	5	-
B	-	4	-
C	1	17	-
D	2	1	-

Table 6.10: Pull request size distribution in Phase 2

Project	Small	Medium	Large
A	4	3	2
B	4	0	1
C	14	6	6
D	5	-	-

Project D was dominated by bug-fix work, while Projects B and C were dominated by feature work. Project A included a mix of tasks. As in Phase 1, smaller pull requests appeared more often in bug-fix work, while larger pull requests appeared more often in feature work. These differences affected flow time and verification effort, and the results are interpreted within the project context.

KPI 1: AI Usage

AI usage increased slightly compared to Phase 1. High AI usage appeared more often, especially in feature-related work. One project contained only high AI usage observations, while low usage was less common overall. Table 6.11 summarizes the AI usage distribution.

Table 6.11: AI usage distribution in Phase 2

Project	None	Low	Medium	High	Total
A	2	2	3	0	7
B	0	0	0	4	4
C	6	1	5	6	18
D	0	0	1	1	2

AI usage appears more integrated into everyday work. However, usage still depends on task type and project context.

KPI 2: Lead Time and PR Flow

The same pattern appeared as in Phase 1. In this study, the coding stage was measured as the time from the first commit on the pull request branch to pull request creation, based on GitHub timestamps. Coding stages were often short, while review stages remained much longer and dominated the pull request lifecycle.

Table 6.12 illustrates the Phase 2 pattern using Project C, where several large or complex changes create long total durations.

Table 6.12: Development flow in Phase 2 (Project C)

AI Level	Coding (h)	Review (h)	Total (h)
None	1.6	164.4	166.0
Low	0.13	292.6	292.7
Medium	195.8	2791.0	2986.8
High	67.5	174.7	242.2

Several pull requests exhibited unusually long flow times. These were typically associated with large or complex changes. In this analysis, task size was approximated using pull

request size, measured as the total number of added and deleted lines. These outliers affected averages. Lead time did not vary consistently with AI usage alone. Pull request size and task complexity remained the main factors.

KPI 3: Verification Effort

Verification effort varied more than in Phase 1. A few projects had very low rework, while others had very high rework, especially for large feature tasks. Table 6.13 presents this pattern in Project C.

Table 6.13: Verification results in Phase 2 (Project C)

AI	Rework	Rounds
None	~20%	2
Low	~100%	5
Medium	~100%	10+
High	~75%	3

Verification effort was observed across all AI usage levels. Pull requests with AI still required review and correction. Higher verification effort was mainly linked to larger pull requests. Small and well-scoped changes required less rework.

KPI 4: Defect Escape

No defect escape signals were observed in Phase 2. This differs from Phase 1. The result may reflect stronger verification, stabilization after earlier changes, or a short observation window. Table 6.14 summarizes the defect signals.

Table 6.14: Defect signals in Phase 2

Project	Rate	Context
A	0%	-
B	0%	-
C	0%	-
D	0%	-

KPI 5: Developer Experience

The second survey received 12 responses. Participation in AI-related training increased slightly compared to Phase 1. Seven participants reported no formal training, while two reported Silver-level training and three reported Gold-level training. GitHub Copilot remained the main tool, and a few developers also used Cursor.

AI was used more broadly across tasks in Phase 2. In addition to coding, developers reported using AI tools for debugging, understanding code, testing, and documentation. Survey responses suggest that AI tools helped developers continue work and learn new parts of the system. At the same time, new issues appeared, including waiting time for responses, interruptions in focus, increased workload, and fatigue. Trust in AI tools remained moderate, and developers continued to rely on their own judgment and remain responsible for decisions.

Open-ended responses reinforced this pattern. Developers reported faster task completion and greater output within the same time, but they also described fatigue, interruptions, and task switching while waiting for AI-generated responses. AI tools continued to support exploration and planning, but they appeared less effective in complex system contexts.

Combined Interpretation

Phase 2 confirmed the main pattern from Phase 1. Coding stages were often short. However, review and verification still took most of the time. Verification effort depended mainly on task size and complexity.

New signals appeared in developer experience. Faster work was associated with more task switching, higher cognitive load, and less stable focus. AI changed how work was done, but review and validation remained necessary.

6.3 Cross-Phase Comparison

The results from both phases indicate a similar overall pattern. Coding stages were often short, while review stages took more time and remained the dominant part of the pull request lifecycle in both phases.

AI usage increased slightly in Phase 2. High AI usage appeared more often, especially in

feature-related work. However, this did not change the overall flow pattern.

Verification effort was present in both phases. Many pull requests required changes after review had started. In Phase 2, the variation was larger. A few projects had very low rework, while others had high rework. This difference was mainly linked to task size and complexity.

Defect signals appeared in Phase 1 but not in Phase 2. This may reflect improved verification or a shorter observation window in the second phase.

Developer experience remained mostly stable. AI tools supported coding, learning, and understanding in both phases. At the same time, Phase 2 indicated additional effects, including waiting time, interruptions, and increased workload.

Across both phases, task complexity had a strong influence on the results. Large pull requests took more time and required more verification. Overall, the results indicate a consistent pattern. When using AI tools, developers produced code faster. However, review and validation still required time. Effort appeared to shift from initial coding toward review and verification rather than being eliminated.

7 Discussion

This chapter discusses the findings of the literature review and industrial case study in relation to the research questions and prior work on AI-assisted software development productivity.

7.1 Overview of Findings

The findings suggest a consistent pattern in AI-assisted software development. AI tools often help developers produce initial code more quickly, especially for routine and well-scoped tasks. However, review and verification remain dominant parts of the development process, and rework continues to appear across many pull requests.

Task complexity strongly affects the observed patterns. Larger and more complex changes require more review effort, longer coordination, and additional validation regardless of AI usage level. The survey results further indicate that AI tools support learning, task execution, and developer flow in some situations, while also introducing interruptions, waiting time, and increased cognitive load.

Overall, the findings suggest that AI tools redistribute effort rather than remove it. Initial implementation effort may decrease, but verification, coordination, and review effort remain essential parts of software development work.

7.2 RQ1: How is Productivity Measured in Software Engineering?

The literature indicates that productivity is still primarily measured using efficiency-related metrics, such as development speed, throughput, and output. These measures focus on observable activity and support delivery tracking and planning. Frameworks such as DORA, SPACE, and DX extend this perspective by incorporating additional dimensions related to system stability, collaboration, and developer experience. However, efficiency-related indicators remain dominant in both research and practice.

The findings also suggest that productivity measurement in the AI-assisted development era is still evolving. Most proposed AI-related metrics build on existing frameworks rather than introducing entirely new measurement models. As a result, many organizations continue to rely on pre-AI productivity concepts adapted to AI-assisted workflows.

7.3 RQ2: What Limitations do Existing Productivity Metrics Have in AI-Assisted Development?

The findings of this study suggest that traditional productivity measures are insufficient for evaluating AI-assisted software development. Metrics focused primarily on output, throughput, or coding speed do not capture the broader effects of AI usage on verification effort, review processes, or developer experience.

While AI tools may accelerate initial implementation, this does not necessarily reduce overall effort. Review, coordination, and validation remain necessary, particularly for larger or more complex tasks. Productivity, therefore, cannot be evaluated solely through speed or output measures.

The results support a multi-dimensional view of productivity that includes efficiency, effectiveness, and sustainability. In AI-assisted development, productivity depends not only on how quickly code is produced, but also on how reliably it can be reviewed, maintained, validated, and integrated into existing systems.

7.4 RQ3: How Can Existing Frameworks Support Productivity Measurement in AI-Assisted Workflows?

Existing frameworks provide useful elements for measuring productivity. The DORA framework focuses on delivery performance and captures flow and stability. The SPACE framework includes developer experience and captures factors such as focus, satisfaction, and collaboration. DX frameworks also consider developer experience and work conditions.

The results indicate that these frameworks remain relevant. However, they do not fully address AI-assisted development. In this study, elements from these frameworks were combined. Repository data captures flow and quality signals, while survey data captures

developer experience. This combination allows a more complete view of productivity by integrating technical workflow signals with developer experience data.

7.5 RQ4: How Can Productivity Indicators be Operationalized in a Real Software Development Environment?

The study applies a small set of indicators in an industrial setting. The indicators measure AI usage, development flow, verification effort, defect signals, and developer experience. The findings suggest that these indicators can be collected using existing tools. No intrusive employee monitoring is required. The indicators provide useful signals about how work is distributed across coding, review, and validation. However, results require context. Interpretation depends on task type, pull request size, and project structure. For this reason, the indicators should be used together rather than in isolation.

From an organizational perspective, the framework also makes previously implicit work visible and measurable. Representatives from Basware noted that several findings aligned with existing practical intuitions about review bottlenecks, pull request size, and verification effort. However, the framework transformed these observations from informal assumptions into observable indicators that supported more structured discussion about workflow patterns and development practices across projects.

Overall, the case study indicates that it is feasible to operationalize multi-dimensional productivity measurement in a real development setting using existing development tooling and lightweight developer feedback mechanisms.

7.6 RQ5: What Insights Emerge from Applying These Indicators in Practice?

The application of the indicators in a real development environment provides several practical insights. The findings suggest that the primary productivity bottleneck in AI-assisted development may shift from code creation toward review, validation, and coordination. While AI tools accelerate initial implementation, review stages often remain the dominant part of the pull request lifecycle.

Importantly, review-stage duration in this study refers mainly to calendar time rather than directly measured human effort. Long review stages may include waiting time, reviewer availability, coordination delays, or active review work. This distinction suggests that improving productivity may require organizational and workflow improvements in addition to technical tooling improvements.

Task complexity also strongly influences outcomes. Larger pull requests are associated with longer review times, higher rework, and increased verification effort, while smaller changes support more stable and manageable workflows. The findings therefore support the use of smaller pull requests and more incremental development practices in AI-assisted environments.

The results also highlight the continuing importance of human oversight. Verification effort remained high across AI usage levels, and review work likely included not only functional corrections but also stylistic, architectural, and maintainability-related adjustments. This suggests that part of the verification burden relates to aligning generated code with team conventions, system design principles, and long-term maintainability requirements.

The findings further indicate that effective use of AI tools depends on organizational practices and developer capability. AI-assisted development may reduce effort for repetitive and well-scoped tasks, such as generating unit tests, but this does not necessarily mean that sound engineering practices are followed. Structured guidance on prompting, verification, review standards, and workflow conventions appears increasingly important for maintaining quality and consistency.

Developer experience also remains an important part of productivity. Survey responses indicate that AI tools can support learning, exploration, and flow, while also introducing interruptions, waiting time, and cognitive load. Trust in AI-generated output remained moderate rather than complete, suggesting that developers continue to rely heavily on personal judgment and validation.

These findings also reflect characteristics of the organizational context. Basware is an established product company with existing systems, processes, and review practices. In such environments, AI-assisted development interacts with existing coordination structures rather than replacing them. Shared ownership, architectural complexity, and established workflows increase the need for validation and coordination compared to smaller or greenfield environments.

Overall, the findings suggest that AI-assisted development changes the structure of soft-

ware engineering work rather than simply increasing productivity. As implementation becomes faster, evaluation, coordination, review, and integration become increasingly important parts of the developer role.

7.7 Comparison with Related Work

The findings of this study align with several observations reported in prior literature. Previous studies have shown that AI tools can increase coding speed, particularly for routine and well-scoped tasks. The case study supports this pattern, as coding stages were often relatively short when AI tools were used.

At the same time, the findings indicate that faster code generation does not eliminate the need for review and validation. This is consistent with earlier work emphasizing the importance of verifying AI-generated code and maintaining human oversight in development workflows.

However, the present study also contrasts with some earlier studies reporting reduced review time in AI-assisted development. In the analyzed organizational setting, review-related effort and review-stage duration often remained high or increased, particularly for larger or more complex changes. These findings suggest that AI-assisted development may shift effort from implementation toward validation, coordination, and integration rather than reducing overall workflow effort.

Compared to controlled experimental studies reporting large efficiency gains, the results of this study are more moderate. This difference may be explained by the inclusion of real organizational constraints, task complexity, review practices, and coordination overhead present in industrial software development environments.

This study also addresses several gaps identified in the literature review. Previous research has often examined productivity through isolated perspectives, such as coding speed, repository activity, code quality, or developer perception, separately. In contrast, this study combines repository telemetry, workflow indicators, and developer experience data within the same measurement framework. The framework also operationalizes verification-related indicators, such as post-review rework and review-stage effort, which earlier literature frequently discussed conceptually but rarely measured in practical organizational settings.

Overall, the findings support existing research while extending it through a multi-dimensional

and practice-oriented evaluation of productivity in AI-assisted software development.

7.8 Implications for Productivity Measurement

This study supports measuring productivity as a balance between efficiency, effectiveness, and sustainability rather than through speed or output alone. Faster code generation does not necessarily imply higher overall productivity if verification effort, coordination overhead, or maintainability concerns increase at the same time. The AI Productivity Triangle provides a structure for organizing these interacting dimensions.

The findings suggest that organizations should combine repository telemetry with developer feedback in order to observe both technical workflow outcomes and developer experience. Productivity measurement should focus on patterns and trade-offs over time rather than isolated metrics.

The study also highlights the importance of traceability in AI-assisted development. Representatives from the case organization noted that a stronger linkage between pull requests, Jira issues, and follow-up defects would improve visibility into how changes evolve and how failures originate. This becomes increasingly important when AI-generated code is involved.

The findings further suggest that software quality should not be evaluated only through defect counts or short-term correctness. Maintainability and long-term system control remain critical concerns. AI-generated code may satisfy immediate functional requirements while still introducing architectural inconsistency, unclear abstractions, or patterns that reduce confidence in future modification and extension of the system.

From a practical perspective, the results support smaller and more incremental pull requests. Larger changes were associated with longer review times, higher rework, and increased verification effort. Since review becomes a dominant part of the workflow, organizations may also need clearer review expectations, better distribution of review responsibilities, and more predictable review cycles.

An additional practical concern relates to the transparency and debuggability of AI-generated code. AI-generated solutions may sometimes favor defensive error-handling patterns that obscure the original source of failures or complicate root-cause analysis. This increases the importance of review practices, architectural conventions, and explicit engineering guidelines when integrating AI-generated code into production systems.

Developer training also appears increasingly important. Structured guidance on prompting, verification, testing, and appropriate use cases may improve both efficiency and quality. In addition, organizations should monitor sustainability-related factors such as interruptions, cognitive load, workload, and trust in AI tools to ensure that productivity gains do not come at the expense of developer well-being.

Overall, the findings indicate that AI-assisted development changes the structure of software engineering work rather than simply accelerating it. As implementation effort decreases, the relative importance of review, validation, coordination, and workflow management increases.

7.9 Future Directions for AI-Assisted Software Development

AI-assisted software development is still evolving rapidly. Current tools appear strongest in routine coding tasks, information search, and initial solution generation. Review, validation, and integration remain largely human responsibilities. In the short term, AI tools are becoming an increasingly common part of everyday software development work. They can accelerate coding and reduce friction in routine tasks, but they may also increase pressure on review capacity, governance practices, and quality assurance processes as organizations adapt to AI-assisted workflows.

In the medium term, AI assistance is already expanding beyond code generation toward testing, refactoring, review support, and workflow coordination. As these capabilities mature, productivity gains may depend less on faster implementation alone and more on improvements across the broader software development process.

In the longer term, AI-assisted development may further shift the role of developers toward architecture, verification, prioritization, coordination, and decision-making. In such environments, the central productivity question may no longer be how quickly code is produced, but how effectively human expertise and AI capabilities are combined to produce reliable, maintainable, and valuable software.

From the perspective of this study, organizations that treat AI tools only as speed tools may see limited benefits. Long-term gains may depend more on governance, workflow redesign, and engineering practices for human-AI collaboration than on tool capability alone.

7.10 Validity and Reliability

This study has limitations that affect the validity and reliability of the findings. The discussion of validity and reliability follows the case study research quality criteria [50].

Construct validity refers to whether the selected indicators capture productivity. In this study, productivity was measured using repository data, Jira data, and a developer survey. This provides a broader view than single metrics, but it still does not capture all aspects of development work. For example, the coding stage was measured from the first commit to pull request creation. Work done before the first commit is not included. AI tool usage was also self-reported in Jira and may not reflect all actual usage. For this reason, the indicators should be seen as approximations of productivity.

Internal validity refers to whether the observed patterns are caused by AI tool usage or by other factors. This study does not use a controlled experimental setup, so causal conclusions cannot be made. Factors such as pull request size, task type, project context, and review practices may influence the results. The analysis considers these factors by interpreting results within each project and by using multiple indicators. Even so, the findings describe patterns rather than direct cause-and-effect relationships.

External validity refers to how well the findings apply to other contexts. The study was conducted in one organization with existing systems and processes. These conditions affect how AI tools are used and how work is reviewed. The findings are therefore most relevant to similar environments. Different results may appear in smaller teams or in new projects with simpler systems.

Reliability refers to whether the study could be repeated with similar findings. Data collection was based on scripts for GitHub and Jira, which help keep the process consistent. The survey was also reviewed before use. However, the observation period is short, and the dataset is limited. AI tool usage is self-reported, which may vary between developers. If the study is repeated, the exact numbers may change, even if the overall patterns remain similar.

The findings were also discussed in a final meeting with representatives from Basware, who considered them broadly consistent with practical experience in the organization. In addition, the thesis was reviewed by the primary Basware contact responsible for coordinating the study from the company side. Together, these steps strengthen the credibility and practical relevance of the case study findings.

Overall, the study provides a useful view of productivity in AI-assisted development in this context. The findings should be interpreted as indicative patterns, not as general or causal conclusions.

8 Conclusions

This thesis examined how productivity can be measured in AI-assisted software development by combining a literature review with a case study conducted in an industrial setting.

The thesis introduced the AI Productivity Triangle, which describes productivity as a balance between efficiency, effectiveness, and sustainability. Based on this model, a measurement framework was defined and applied using repository data and developer surveys.

The results indicate that AI-assisted development changes how effort is distributed. Initial code production is often faster, but verification, review, and rework remain important parts of the process. Developer experience is also important. AI tools support learning and task execution, but they can also introduce interruptions and increased workload. Together, these results suggest that productivity in AI-assisted development cannot be evaluated through speed alone.

This thesis makes three main contributions. First, it proposes a conceptual model for understanding productivity in AI-assisted development. Second, it defines a set of practical indicators that combine repository data and developer experience data. Third, it provides empirical evidence from an industrial case study, suggesting that AI tools appear to shift effort from coding toward verification, coordination, and review work.

The findings have practical implications for organizations adopting AI tools. Measuring coding speed alone may create a misleading view of productivity. Organizations should also monitor review effort, quality outcomes, and developer experience. Productivity gains are more likely when tools, workflows, and team practices develop together.

Future work can extend this research in several ways. Longer observation periods would support analysis of long-term effects, such as quality and stability. Studies across multiple organizations would improve generalizability. More detailed measurement of AI usage could provide deeper insight into how different usage patterns affect productivity. In addition, further work can refine the measurement framework and examine human factors, including cognitive load, learning, and work practices in more detail.

Overall, the findings suggest that AI tools do not simply increase productivity. Instead, they change the structure of software development work. The central challenge is no longer

only to write code faster, but to combine human judgment and AI capability in an effective and sustainable way.

Bibliography

- [1] S. Wagner and F. Deissenboeck. “Defining Productivity in Software Engineering”. In: *Rethinking Productivity in Software Engineering*. Ed. by C. Sadowski and T. Zimmermann. Apress, 2019, pp. 29–38. DOI: [10.1007/978-1-4842-4221-6_4](https://doi.org/10.1007/978-1-4842-4221-6_4).
- [2] Google Cloud. *DORA’s Software Delivery Metrics: The Four Keys*. <https://dora.dev/guides/dora-metrics-four-keys/>. Accessed: 15.10.2025. 2023.
- [3] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck, and J. Butler. “The SPACE of Developer Productivity: There’s More to It Than You Think”. In: *ACM Queue* 19.1 (2021), pp. 20–48. DOI: [10.1145/3453928](https://doi.org/10.1145/3453928).
- [4] DX. *Measuring Developer Productivity with the DX Core 4*. <https://getdx.com/research/measuring-developer-productivity-with-the-dx-core-4/>. Accessed: 15.10.2025. 2023.
- [5] A. Mohamed, M. Assi, and M. Guizani. *The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Literature Review*. 2025. DOI: [10.48550/arXiv.2507.03156](https://doi.org/10.48550/arXiv.2507.03156). arXiv: [2507.03156](https://arxiv.org/abs/2507.03156) [cs.SE].
- [6] V. Stray, A. Barbala, and V. T. Wivestad. “Human–AI Collaboration in Software Development”. In: *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 2025, pp. 1325–1332. DOI: [10.1145/3696630.3730566](https://doi.org/10.1145/3696630.3730566).
- [7] T. Ekandjo. *Human–AI Interaction in Regulating Productivity and Wellbeing*. 2024. DOI: [10.26686/wgtn.25377481](https://doi.org/10.26686/wgtn.25377481).
- [8] A. R. Hevner, S. T. March, J. Park, and S. Ram. “Design Science in Information Systems Research”. In: *MIS Quarterly* 28.1 (2004), pp. 75–105. DOI: [10.2307/25148625](https://doi.org/10.2307/25148625).
- [9] K. Peppers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. “A Design Science Research Methodology for Information Systems Research”. In: *Journal of Management Information Systems* 24.3 (2007), pp. 45–77. DOI: [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302).
- [10] F. Almeida and P. Carneiro. “Perceived Importance of Metrics for Agile Scrum Environments”. In: *Information* 14.6 (2023), p. 327. DOI: [10.3390/info14060327](https://doi.org/10.3390/info14060327).

- [11] M. Sallin, M. Kropp, C. Anslow, J. W. Quilty, and A. Meier. “Measuring Software Delivery Performance Using the Four Key Metrics of DevOps”. In: *Agile Processes in Software Engineering and Extreme Programming*. Ed. by P. Gregory, C. Lassenius, X. Wang, and P. Kruchten. Springer International Publishing, 2021, pp. 103–119. DOI: [10.1007/978-3-030-78098-2_7](https://doi.org/10.1007/978-3-030-78098-2_7).
- [12] DORA. *DORA Research: 2022*. <https://dora.dev/research/2022/>. Accessed: 10.05.2026. 2022.
- [13] Findy Team. *Real-World Success Stories: How Organizations Excelled with DORA and SPACE Frameworks*. <https://www.linkedin.com/pulse/real-world-success-stories-how-organizations-excelled-s7w8c/>. Accessed: 10.05.2026. 2024.
- [14] DX. *DX Core 4: How 300+ Companies Measure Developer Productivity*. <https://byteiota.com/dx-core-4-how-300-companies-measure-developer-productivity/>. Accessed: 10.05.2026. 2024.
- [15] “Software Engineering Economics”. In: *Guide to the Software Engineering Body of Knowledge*. Ed. by P. Bourque and R. E. Fairley. IEEE Computer Society, 2014. Chap. 15, pp. 422–423.
- [16] B. Kitchenham. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Tech. rep. EBSE, 2007.
- [17] N. R. Haddaway, M. J. Page, C. C. Pritchard, and L. A. McGuinness. “PRISMA2020: Flow Diagram Tools”. In: *Campbell Systematic Reviews* 18 (2022), e1230. DOI: [10.1002/c12.1230](https://doi.org/10.1002/c12.1230).
- [18] A. Solanke. “Software Development in the Age of AI: Quantifiable Frameworks for Measuring Enhanced Developer Productivity”. In: *International Journal of Current Science Research and Review* (2024), pp. 234–250.
- [19] P. Bose. “Beyond Productivity: A Framework for Measuring Human–AI Synergy in the Software Development Lifecycle (AI-Augmented Productivity Metrics Framework)”. In: *International Journal of Artificial Intelligence, Data Science, and Machine Learning* 6.3 (2025), pp. 43–46.
- [20] S. Bağrıyanık and E. Ersoy. “Measuring AI Impact on Software Engineering Productivity and Quality”. In: *International Conference on Computer Science and Engineering*. 2025, pp. 1187–1192. DOI: [10.1109/UBMK67458.2025.11206890](https://doi.org/10.1109/UBMK67458.2025.11206890).

- [21] C. Symons. “The COSMIC Method for Measuring the Work-Output Component of Productivity”. In: *Rethinking Productivity in Software Engineering*. Ed. by C. Sadowski and T. Zimmermann. Apress, 2019, pp. 191–204. DOI: [10.1007/978-1-4842-4221-6_17](https://doi.org/10.1007/978-1-4842-4221-6_17).
- [22] Z. Cui, M. Demirer, S. Jaffe, L. Musolff, S. Peng, and T. Salz. *The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers*. <https://www.microsoft.com/en-us/research/publication/the-effects-of-generative-ai-on-high-skilled-work-evidence-from-three-field-experiments-with-software-developers/>. Accessed: 20.10.2025. 2025.
- [23] A. Kumar, V. Khare, D. Sharma, S. Kumar, V. Saini, A. Yadav, S. Jain, A. Rana, P. Verma, V. Meena, and A. Edubilli. *Measuring AI’s True Impact on Developer Productivity*. 2025. DOI: [10.48550/arXiv.2509.19708](https://doi.org/10.48550/arXiv.2509.19708).
- [24] R. Pandey, P. Singh, R. Wei, and S. Shankar. *Transforming Software Development: Evaluating the Efficiency and Challenges of GitHub Copilot in Real-World Projects*. 2024. DOI: [10.48550/arXiv.2406.17910](https://doi.org/10.48550/arXiv.2406.17910).
- [25] T. Weber, M. Brandmaier, A. Schmidt, and S. Mayer. “Significant Productivity Gains through Programming with Large Language Models”. In: *Proceedings of the ACM on Human-Computer Interaction* 8.EICS (2024), pp. 1–29. DOI: [10.1145/3661145](https://doi.org/10.1145/3661145).
- [26] P. Häkkinen. “Generative AI and Software Developer Productivity: A Pilot Randomized Controlled Trial”. Master’s thesis. MA thesis. Aalto University School of Business, 2025.
- [27] F. Song, A. Agarwal, and W. Wen. *The Impact of Generative AI on Collaborative Open-Source Software Development*. 2025. DOI: [10.48550/arXiv.2410.02091](https://doi.org/10.48550/arXiv.2410.02091).
- [28] J. Becker, N. Rush, E. Barnes, and D. Rein. *Measuring the Impact of Early-2025 AI on Developer Productivity*. 2025. DOI: [10.48550/arXiv.2507.09089](https://doi.org/10.48550/arXiv.2507.09089).
- [29] S. Sonkar. “Boosting Developer Efficiency with AI: A Technical Review”. In: *Saracouncil Journal of Engineering and Computer Sciences* 4.9 (2025), pp. 500–508.
- [30] D. Russo. “Navigating the Complexity of Generative AI Adoption in Software Engineering”. In: *ACM Transactions on Software Engineering and Methodology* 33.5 (2024), pp. 1–50. DOI: [10.1145/3652154](https://doi.org/10.1145/3652154).

- [31] P. Hoffmann, D. Zercher, and T. C. Schimmer. *How AI Communication Capabilities Affect Developer Productivity in AI Pair Programming: Insights From a Large Software Development Organization*. https://aisel.aisnet.org/treos_ecis2025/23. Accessed: 20.10.2025. 2025.
- [32] O. Hilska. *Measuring the Productivity Impact of AI Coding Tools*. <https://www.swarmia.com/blog/productivity-impact-of-ai-coding-tools/>. Accessed: 20.10.2025. 2025.
- [33] Microsoft. *Secure AI Productivity: Preparing Organizations for AI at Work*. Conference session presented at the Microsoft Copilot and AI Agent Summit. Accessed: 23.11.2025. 2025.
- [34] M. Coutinho, L. Marques, A. Santos, M. Dahia, C. Franca, and R. de Souza Santos. *The Role of Generative AI in Software Development Productivity: A Pilot Case Study*. 2024. DOI: [10.48550/arXiv.2406.00560](https://doi.org/10.48550/arXiv.2406.00560).
- [35] J. D. Weisz, S. Kumar, M. Muller, K.-E. Browne, A. Goldberg, E. Heintze, and S. Bajpai. “Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise”. In: *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 2025. DOI: [10.1145/3706599.3706670](https://doi.org/10.1145/3706599.3706670).
- [36] A. Valiulla. *An Empirical Study of Measurement Framework Adoption - DORA and SPACE: How Organizational Context Shapes Success and Failure*. 2025. DOI: [10.2139/ssrn.5344643](https://doi.org/10.2139/ssrn.5344643).
- [37] S. Li, Y. Cheng, J. Chen, J. Xuan, S. He, and W. Shang. “Assessing the Performance of AI-Generated Code”. In: *IEEE International Symposium on Software Reliability Engineering*. 2024, pp. 216–227. DOI: [10.1109/ISSRE62328.2024.00030](https://doi.org/10.1109/ISSRE62328.2024.00030).
- [38] S. Bandaru. *Is the Role of DORA Metrics Still Relevant in an Era of AI Coding?* <https://www.hivel.ai/blog/dora-metrics-in-ai-coding>. Accessed: 16.10.2025. 2025.
- [39] Microsoft. *Copilot in Action: Unlocking AI-Powered Productivity*. Conference session presented at the Microsoft Industry Summit. Accessed: 23.11.2025. 2025.
- [40] R. Cariaga. *Human–AI Collaboration in Decision-Making, Creativity, and Productivity: Systematic Review*. 2025. DOI: [10.2139/ssrn.5378095](https://doi.org/10.2139/ssrn.5378095).

- [41] MIT NANDA. *The GenAI Divide: State of AI in Business 2025*. https://mlq.ai/media/quarterly_decks/v0.1_State_of_AI_in_Business_2025_Report.pdf. MIT NANDA report Accessed: 10.10.2025. 2025.
- [42] Z. Yi and S. Ayangbah. “The Impact of AI Innovation Management on Organizational Productivity and Economic Growth”. In: *International Journal of Business Management and Economic Review* (2024).
- [43] B. L. Pearson. *SPACE Metrics Framework for Developers Explained*. <https://linearb.io/blog/space-framework>. Accessed: 20.10.2025. 2025.
- [44] T. Bruneaux. *The SPACE Framework: A Complete Guide to Developer Productivity*. <https://space-framework.com/>. Accessed: 16.10.2025. 2025.
- [45] B. Doerrfeld. *How DX Core 4 Aims to Unify Developer Productivity Frameworks*. <https://leaddev.com/reporting/dx-core-4-aims-to-unify-developer-productivity-frameworks>. Accessed: 17.10.2025. 2024.
- [46] DX. *Measuring AI Code Assistants and Agents: The DX AI Measurement Framework*. <https://getdx.com/research/measuring-ai-code-assistants-and-agents/>. Accessed: 17.10.2025. 2024.
- [47] S. Sikand, K. Kaur, V. S. Sharma, K. Singi, V. Kaulgud, T. Tung, P. Sharma, and A. Burden. “How Much SPACE Do Metrics Have in GenAI Assisted Software Development?” In: *Proceedings of the 17th Innovations in Software Engineering Conference*. 2024, pp. 1–5. DOI: [10.1145/3641399.3641419](https://doi.org/10.1145/3641399.3641419).
- [48] Y. Denisov-Blanch. *Does AI Actually Boost Developer Productivity?* https://youtu.be/tbDDYKRFjkh?si=esJr_8edIZIR7yjq. Stanford talk- YouTube, Accessed: 17.10.2025. 2024.
- [49] A. Masood. *Rethinking Developer Productivity in the Age of AI: Metrics That Actually Matter*. <https://medium.com/@adnanmasood/rethinking-developer-productivity-in-the-age-of-ai-metrics-that-actually-matter-61834691c76e>. Accessed: 20.10.2025. 2025.
- [50] R. K. Yin. *Case Study Research: Design and Methods*. SAGE Publications, 2009.

Appendix A Included Sources

The following Table A.1 lists the 34 sources included in the literature review. The sources include academic papers, preprints, theses, and industry reports.

Table A.1: Overview of included sources in the literature review

Reference Nr	Authors	Title	Type
[24]	Pandey et al.	Transforming Software Development: Evaluating the Efficiency and Challenges of GitHub Copilot in Real-World Projects	Preprint
[25]	Weber et al.	Significant Productivity Gains through Programming with Large Language Models	Academic
[26]	Häkkinen	Generative AI and Software Developer Productivity: A Pilot Randomized Controlled Trial	Thesis
[5]	Mohamed et al.	The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Literature Review	Preprint
[6]	Stray et al.	Human–AI Collaboration in Software Development: A Mixed-Methods Study of Developers’ Use of GitHub Copilot and ChatGPT	Academic
[7]	Ekandjo	Human–AI Interaction in Regulating Productivity and Wellbeing	Thesis
[18]	Solanke	Software Development in the Age of AI: Quantifiable Frameworks for Measuring Enhanced Developer Productivity	Academic
[19]	Bose	Beyond Productivity: A Framework for Measuring Human–AI Synergy in the Software Development Lifecycle	Academic
[20]	Bağrıyanık and Ersoy	Measuring AI Impact on Software Engineering Productivity and Quality	Academic
[22]	Cui et al.	The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers	Industry Research

[23]	Kumar et al.	Measuring AI's True Impact on Developer Productivity	Preprint
[27]	Song et al.	The Impact of Generative AI on Collaborative Open-Source Software Development	Preprint
[28]	Becker et al.	Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity	Preprint
[29]	Sonkar	Boosting Developer Efficiency with AI: A Technical Review	Academic
[30]	Russo	Navigating the Complexity of Generative AI Adoption in Software Engineering	Academic
[31]	Hoffmann et al.	How AI Communication Capabilities Affect Developer Productivity in AI Pair Programming: Insights From a Large Software Development Organization	Academic
[32]	Hilska	Measuring the Productivity Impact of AI Coding Tools: A Practical Guide for Engineering Leaders	Industry
[33]	Microsoft	Secure AI Productivity: Preparing Organizations for AI at Work	Industry
[34]	Coutinho et al.	The Role of Generative AI in Software Development Productivity: A Pilot Case Study	Preprint
[35]	Weisz et al.	Examining the Use and Impact of an AI Code Assistant on Developer Productivity and Experience in the Enterprise	Academic
[36]	Valiulla	An Empirical Study of Measurement Framework Adoption – DORA and SPACE: How Organizational Context Shapes Success and Failure	Preprint
[37]	Li et al.	Assessing the Performance of AI-Generated Code	Academic
[38]	Bandaru	Is the Role of DORA Metrics Still Relevant in an Era of AI Coding?	Industry
[40]	Cariaga	Human–AI Collaboration in Decision-Making, Creativity, and Productivity: Systematic Review	Preprint
[41]	MIT NANDA	The GenAI Divide: State of AI in Business 2025	Industry

[42]	Yi and Ayangbah	The Impact of AI Innovation Management on Organizational Productivity and Economic Growth	Academic
[43]	Pearson	SPACE Metrics Framework for Developers Explained	Industry
[44]	Bruneaux	The SPACE Framework: A Complete Guide to Developer Productivity	Industry
[45]	Doerrfeld	How DX Core 4 Aims to Unify Developer Productivity Frameworks	Industry
[46]	DX	Measuring AI Code Assistants and Agents: The DX AI Measurement Framework	Industry
[47]	Sikand et al.	How Much SPACE Do Metrics Have in GenAI-Assisted Software Development?	Academic
[48]	Denisov-Blanch	Does AI Actually Boost Developer Productivity?	Industry
[39]	Microsoft	Copilot in Action: Unlocking AI-Powered Productivity	Industry
[49]	Masood	Rethinking Developer Productivity in the Age of AI: Metrics That Actually Matter	Industry

Appendix B Evidence-to-Metrics Mapping

This appendix presents the detailed evidence-to-metrics mapping referenced in Chapter 4. It summarizes how the literature measures productivity in practice. It also presents which signals can be measured in organizational settings. The appendix links these measures to established frameworks such as DORA, SPACE, and DX Core 4. It also considers recent proposals for measuring AI-assisted development.

The structure follows the three dimensions of the AI Productivity Triangle:

- Efficiency
- Effectiveness
- Sustainability

B.1 Efficiency Metrics Mapping (Throughput, Speed, Task Fit)

Research often reports efficiency gains when developers use AI coding tools. However, the size of these gains varies. The results depend on task type, programming language, and workflow integration. Table B.1 summarizes the efficiency-related metrics identified in the reviewed literature.

Framework Alignment (Efficiency)

- DORA (Speed): lead time, deployment frequency
- DX AI: utilization, time saved, ROI perspective
- DX Core 4: engineering speed and delivery

Key implication: Efficiency metrics must be segmented. Important factors include task type, pull request size, workflow integration, and level of AI usage. Raw throughput alone does not indicate whether teams achieve real flow improvement or only redistribute effort

B.2 Effectiveness Metrics Mapping (Quality, Reliability, Rework)

Effectiveness measurement should include more than functional correctness. It should also consider reliability, maintainability, and downstream development cost. Table B.2 summarizes the effectiveness-related metrics identified in the reviewed literature.

Framework Alignment (Effectiveness)

- DORA (Stability): change failure rate, mean time to recovery (MTTR)
- DX Core 4: code quality and maintainability
- SPACE: performance and correctness-related activity

Key implication: Functional correctness alone is not enough. Teams need multiple layers of evaluation. These layers include review signals, CI results, static analysis, and post-merge outcomes. Together, these signals indicate whether AI tools improve effectiveness or shift costs to later stages.

B.3 Sustainability Metrics Mapping (Human Factors, Coordination, Adoption)

Sustainability reflects the human and organizational side of development. It includes developer experience, team coordination, adoption patterns, and long-term workflow stability. Table B.3 summarizes the sustainability-related metrics identified in the reviewed literature

Framework Alignment (Sustainability)

- SPACE: developer experience, satisfaction, and flow
- DX Core 4: developer experience and business impact
- Adoption research: Compatibility $\rightarrow$ Adoption $\rightarrow$ Outcome
- DX AI: utilization, impact, and cost

Key implication: Sustainability metrics help interpret productivity outcomes and indicate whether AI adoption creates lasting benefits or introduces long-term organizational challenges.

B.4 Integrated Evidence Map (Cross-Dimension Summary)

This section summarizes the evidence across studies referenced in Chapter 4. The structure follows the three dimensions of the AI Productivity Triangle.

Table B.4 summarizes the cross-dimension evidence.

Measurement Framework Implications

- DORA and SPACE remain important foundations. However, they do not capture all aspects of AI-assisted development.
- DX Core 4 connects delivery speed and code quality at an organizational level.
- DX AI introduces AI-specific signals such as utilization and cost.
- Collaboration metrics remain limited in current research on generative AI.

Table B.1: Efficiency metrics identified in the literature

Metric Category	Example Measures in Literature	Typical Data Source	AI-Specific Adaptation Needed?	Why It Matters
PR Throughput	PRs per developer-week; merged PRs per month; share merged within 1/3/10 days	Repository telemetry	Yes – segment by AI usage and task type	Detect gross output increase and flow changes
Cycle & Lead Time	PR cycle segmented (coding → review → merge); review duration; stage-time breakdown	PR meta-data, timestamps	Yes – separate coding vs. review stage	Detect coding acceleration vs. downstream verification shift
Task-Specific Time Savings	Time differences for boilerplate vs. architectural tasks; CI/CD vs. debugging	Task labels, PR classification	Yes – segment by task type	Reveal where AI benefits materialize (routine vs. complex work)
AI-Specific Efficiency Telemetry	Acceptance rate; % AI-authored lines; snippet size; time-to-first-useful-edit	IDE/plugin logs or self-report	Yes – required for attribution	Provides AI intensity lens for interpreting speed changes
Onboarding & Comprehension	Time to first PR; time to understand unfamiliar code; AI-assisted comprehension episodes	Repo history + survey	Optional but recommended	Captures indirect efficiency gains via learning acceleration

Table B.2: Effectiveness metrics identified in the literature

Metric Category	Example Measures in Literature	Measures in Literature	Typical Data Source	AI-Specific Adaptation Needed?	Why It Matters
First-Pass Quality	First-pass pass rate; first-pass reviewer approval	CI/test	CI logs; PR reviews	Yes – segment by AI usage	Detect immediate correctness and integration stability
Review-Stage Effectiveness	Reviewer distance; review rounds; request-for-changes frequency	edit review request-frequency	PR review meta-data	Yes – AI-touched PR segmentation	Quantifies verification burden and cleanup effort
Non-Functional Quality	Runtime CPU/memory overhead; performance smells; regressions	delta; static	Profiling tools; static analysis	Recommended	Detect hidden performance and architectural regressions
Stability & Rework	Post-merge work ≤ 14 days; hotfix frequency; bug-issue ratios; change failure rate	re-work	Repo + issue tracker (e.g., Jira)	Yes – AI segmentation required	Identify cost externalization into downstream fixes
Maintainability	Maintainability Index complexity; duplication; violations	delta; duplication; rule	Static analysis tools	Optional but valuable	Assess long-term code health impact

Table B.3: Sustainability metrics identified in the literature

Metric Category	Example Measures in Literature	Measures	Typical Data Source	AI-Specific Adaptation Needed?	Why It Matters
Developer Experience (SPACE)	Satisfaction; cognitive trust; learning support	flow; load; calibration;	Pulse surveys	Yes – AI-focused items	Detect whether gains are sustainable for developers
Verification Burden	Time validating prompt count; time	validation suggestions; iteration cleanup	Survey + PR telemetry	Yes	Identify cognitive and process overhead introduced by AI
Collaboration & Mentorship	Reviewer comment cross-team interaction; peer dialogue	count; volume; interaction;	PR data; network analysis	Optional segmentation	Detect coordination shifts and mentorship impact
Adoption & Governance	Workflow compatibility; policy clarity; shadow AI usage; seniority differences	compatibility; policy clarity; shadow AI usage; seniority differences	Survey + org policy review	Yes	Capture mediators between capability and productivity
Equity Signals	Benefit gaps by seniority; confidence trajectories; peripheral vs. core contributors	gaps by seniority; confidence trajectories; peripheral vs. core contributors	Survey + segmentation	Yes	Detect uneven distribution of benefits or burden

Table B.4: Cross-dimension evidence summary

Dimension	Strong Findings	Mixed / Conditional Findings	Moderators Identified
Efficiency	15–35% faster task completion in enterprise/RCT settings; higher PR throughput	Slower performance in complex OSS tasks; net gains shrink after rework	Task type; codebase maturity; workflow integration; training
Effectiveness	Short-term correctness stable; review approval stable in some contexts	Non-functional regressions common without guardrails	Guardrails; static analysis; profiling; PR size norms
Sustainability	Improved comprehension and learning; flow benefits with IDE integration	Increased verification burden; uneven adoption; trust differences	Workflow compatibility; seniority; governance clarity

Appendix C AI-Assisted Software Development Experience Survey Instrument

This appendix presents the survey instrument used in the Basware pilot study. The survey measured the sustainability dimension of the AI Productivity Triangle. Developers completed the survey once per month. The questions asked about their experience during the previous two weeks.

The survey has four sections: background context, AI tool usage, frequency of AI assistance across tasks, and experience statements measured with a 7-point Likert scale. The survey also includes one optional open-ended reflection question.

C.1 Background and Context

Current Level in AI Training Program

Please indicate your current level in the AI training program:

- Not enrolled / Not started
- Silver
- Gold

AI Tool(s) Used (Past 2 Weeks)

Which AI tool(s) did you use?

- GitHub Copilot
- Cursor
- Claude / Claude Code
- Other (please specify)

Multiple selections were allowed.

C.2 Frequency of AI Assistance by Task Type (Past 2 Weeks)

Participants were asked:

In the last two weeks, how often did you use AI assistance for the following activities?

Table C.1 summarizes the activities included in this question.

Table C.1: Frequency of AI assistance by development activity

Activity	Frequency
Implementing or modifying code (new features/changes)	Not at all - Very often
Understanding existing code (mine or others')	Not at all - Very often
Debugging and troubleshooting (in an existing codebase)	Not at all - Very often
Testing and verification (e.g., writing tests, analyzing failures, validating behavior)	Not at all - Very often
Documentation (e.g., writing or improving docs/comments/READMEs)	Not at all - Very often
Design and architecture (e.g., solution approach, APIs, system structure)	Not at all - Very often
Code review support (e.g., reviewing, explaining, summarizing changes)	Not at all - Very often

Note: The response scale used the options *Not at all*, *Rarely*, *Sometimes*, *Often*, *Very often*.

C.3 Experience Constructs (7-Point Likert Scale)

Participants were instructed:

Thinking about the past two weeks, please indicate how much you agree with the following statements. All statements refer to your experience when using AI assistance (compared to similar work without AI).

Table C.2 presents the survey statements and their corresponding themes.

Table C.2: Experience constructs used in the developer survey

Item Theme	Statement
Verification Ease	When using AI assistance, it is easy for me to check that the output is correct and safe (e.g., tests, edge cases, security, integration behavior, domain rules).
Rework Effort	The output fits quality expectations with only minor rework (e.g., style, architecture, integration requirements, domain correctness).
Flow	Using AI assistance helps me stay in flow by reducing the need to leave my development environment (e.g., fewer web searches or switches to external documentation/tools).
Focus	It helps me maintain focus within a task, even when interacting with AI (e.g., prompt crafting, re-reading or validating output).
Learning	It helps me understand unfamiliar parts of the code-base or problem domain.
Understanding	I understand the AI-assisted solution well enough to confidently modify or extend it.
Guidance	It helps me get started and structure my approach to a task (e.g., design, API usage, implementation strategy).
Clarity	It helps me compare alternatives and choose one with confidence.
Trust	In my work context, AI assistance provides suggestions that are correct and useful.
Judgment	I feel confident deciding when AI should not be used (e.g., risky logic, security-sensitive code, unclear behavior).
Meaningfulness	Using AI assistance makes my work feel more meaningful.
Self-Confidence	It increases my confidence in handling demanding tasks.

Item Theme	Statement
Sense of Control	Even when using AI assistance, I feel that I remain in control and understand what I am responsible for.

Note: The response scale used the options *Strongly disagree*, *Disagree*, *Slightly disagree*, *Neither agree nor disagree*, *Slightly agree*, *Agree*, *Strongly Agree*.

C.4 Optional Open-Ended Reflection

Participants were invited to provide qualitative input:

Is there an example or situation you would like to share that illustrates how using AI assistance has affected your work?

Participants could describe positive effects, challenges, trade-offs, or any other relevant experiences. Respondents were encouraged to reflect on a concrete situation from the past two weeks.

C.5 Construct Mapping to Sustainability Dimensions

For analytical purposes, survey items were grouped into higher-level sustainability clusters. Table C.3 summarizes this mapping.

Table C.3: Mapping of survey items to sustainability dimensions

Sustainability Dimensions	Survey Items
Verification burden	Verification Ease, Rework Effort
Flow and cognitive continuity	Flow, Focus
Learning and capability development	Learning, Understanding, Guidance, Clarity
Trust calibration	Trust, Judgment
Engagement and autonomy	Meaningfulness, Self-Confidence, Sense of Control

Appendix D Phase 1 Detailed Results

This appendix provides detailed project-level results supporting the Phase 1 summary presented in Chapter 6. The appendix includes full tables for development flow, verification effort, defect signals, and developer survey results.

D.1 Observation Context

Phase 1 covers the period from February 9 to March 2. The analysis includes four projects. The data combines repository metrics, Jira issue data, and survey responses.

D.2 Dataset Overview

Filtering was applied as described in Chapter 6. Pull requests without a Jira key were excluded, and pull requests sharing the same Jira key were excluded from AI-segmented analysis.

After filtering, 75 pull requests were merged, 61 pull requests were linked to Jira, and 49 pull requests were used for AI-segmented analysis.

D.3 Project Context

The projects differ in task type and size of changes. These differences affect development flow and verification effort. Projects A and D mainly include bug-fix work, while Project C mainly includes feature work. Project B includes mixed work but has limited data. Smaller pull requests appear more often in bug-fix work, while larger pull requests appear more often in feature work.

Table D.1: Jira issue types in Phase 1

Project	Bug	Story	Other
A	5	0	1
B	1	1	-
C	4	19	-
D	17	10	2

Table D.2: Pull request size distribution in Phase 1

Project	Small	Medium	Large
A	3	2	1
B	2	0	0
C	18	7	6
D	22	9	2

D.4 Detailed KPI Results

D.4.1 KPI 1 - AI Usage

Table D.3 presents the AI usage distribution across projects. Medium AI usage appears most often. High usage appears more often in feature work, while low or no usage appears more often in bug-fix tasks.

Table D.3: AI usage distribution in Phase 1

Project	None	Low	Medium	High	Total
A	2	1	2	1	6
B	0	1	0	1	2
C	2	4	5	6	17
D	2	3	14	5	24

D.4.2 KPI 2 - Lead Time and PR Flow

The following tables present development flow results for each project. Across the projects, coding time is short in most cases, while review time is longer than coding time. Larger pull requests increase total time, and task complexity affects flow more than AI usage.

* large pull request

** long review delay

Table D.4: Development flow in Phase 1 (Project A)

AI Level	Coding (h)	Review (h)	Total (h)
None	0.08	22	22
Low	0.42	4.2	4.6
Medium	34.3*	8.7	43
High	0.01	90**	90

Table D.5: Development flow in Phase 1 (Project B)

AI Level	Coding (h)	Review (h)	Total (h)
Low	0.07	17.8	17.9
High	0.03	2.6	2.7

D.4.3 KPI 3 - Verification Burden

Tables D.8-D.11 summarize verification results for each project. Around half of pull requests require rework. Larger pull requests indicate higher rework, AI usage does not remove verification effort, and review rounds remain fairly stable in many cases.

D.4.4 KPI 4 - Defect Escape

Table D.12 summarizes defect signals in Phase 1. Defect signals appear across projects, and most cases relate to more complex changes.

D.4.5 KPI 5 - Developer Survey

The survey includes 22 responses from developers in the pilot projects.

Training participation

The survey received 22 responses. Most developers had not completed formal AI training: 19 participants reported no training, while one participant reported Silver-level training and two reported Gold-level training.

Table D.6: Development flow in Phase 1 (Project C)

AI Level	Coding (h)	Review (h)	Total (h)
None	0.3	18	19
Low	1.5	29	31
Medium	3.0	35	38
High	6.5	63	69

Table D.7: Development flow in Phase 1 (Project D)

AI Level	Coding (h)	Review (h)	Total (h)
None	0.05	25	25
Low	2.6	22	24.6
Medium	0.8	19	20
High	0.6	19	20

Tool usage

GitHub Copilot is used most often. Cursor is used by some developers, while other tools appear rarely.

AI usage by task

AI is used mainly for writing code, testing, and understanding code. It is used less for design, architecture, and code review.

Experience notes

Developers report better focus and flow in some cases. At the same time, AI outputs require checking, trust in AI remains moderate, and developers keep control of decisions.

Open responses

Open-ended responses indicate that AI helps explore solutions and works well for repetitive tasks. Developers prefer small suggestions, while complex systems remain difficult for AI tools. Incorrect outputs still occur.

Table D.8: Verification in Phase 1 (Project A)

AI	Rework	Rounds
None	~50%	3
Low	~100%	2
Medium	~50%	2
High	0%	2

Table D.9: Verification in Phase 1 (Project B)

AI	Rework	Rounds
Low	0%	1
High	100%	1

D.5 Summary of Phase 1 Detailed Results

The detailed tables support the Phase 1 results presented in Chapter 6. Coding is fast in many cases, review takes more time, rework appears in many pull requests, and larger changes require more effort. AI use appears in many tasks, but it does not remove the need for verification.

Table D.10: Verification in Phase 1 (Project C)

AI	Rework	Rounds
None	~33%	2
Low	~60%	3
Medium	~65%	3
High	~75%	4

Table D.11: Verification in Phase 1 (Project D)

AI	Rework	Rounds
None	~50%	2
Low	~67%	2
Medium	~50%	2
High	~40%	1

Table D.12: Defect signals in Phase 1

Project	Rate	Context
A	~17%	High AI
B	0%	-
C	~22%	Medium / High
D	~10%	Medium

Appendix E Phase 2 Detailed Results

This appendix provides detailed project-level results supporting the Phase 2 summary presented in Chapter 6. The appendix includes full tables for development flow, verification effort, defect signals, and developer survey results.

E.1 Observation Context

Phase 2 covers the period from March 3 to March 20. The analysis includes the same four projects as in Phase 1. The data combines repository metrics, Jira issue data, and survey responses from developers.

E.2 Dataset Overview

Filtering was applied as described in Chapter 6. Pull requests without a Jira key were excluded, pull requests sharing the same Jira key were excluded from AI segmentation, and pull requests with missing AI usage were marked as unknown.

After filtering, 45 pull requests were merged, 35 pull requests were linked to Jira, and 31 pull requests were used for AI-segmented analysis.

E.3 Project Context

Projects differ in task type and size of changes. These differences affect development flow and verification effort. Project D mainly includes bug-fix work. Projects B and C mainly include feature work. Project A includes a mix of tasks. Small pull requests appear more often in bug-fix work, while larger pull requests appear more often in feature work.

E.4 Detailed KPI Results

E.4.1 KPI 1 - AI Usage

Table E.3 presents the AI usage distribution. High AI usage appears more often than in Phase 1, low usage appears less often, and AI use becomes more integrated into development work.

Table E.1: Jira issue types in Phase 2

Project	Bug	Story	Other
A	3	5	-
B	-	4	-
C	1	17	-
D	2	1	-

Table E.2: Pull request size distribution in Phase 2

Project	Small	Medium	Large
A	4	3	2
B	4	0	1
C	14	6	6
D	5	-	-

E.4.2 KPI 2 - Lead Time and PR Flow

The following tables present development flow results for each project. Coding time is short in many cases, but review time remains longer than coding time. Large pull requests create very long durations, outliers affect averages, and task complexity has a stronger effect than AI usage.

* long review delay

** very large pull request

E.4.3 KPI 3 - Verification Burden

Tables E.8-E.11 summarize verification results. Verification effort varies across projects. Larger pull requests indicate higher rework; some projects show very low rework, AI usage does not remove verification effort, and review rounds increase mainly with task complexity.

E.4.4 KPI 4 - Defect Escape

Table E.12 summarizes defect signals. No defect escape signals were observed in Phase 2. This differs from Phase 1, and the result may be affected by the short observation period.

Table E.3: AI usage distribution in Phase 2

Project	None	Low	Medium	High	Total
A	2	2	3	0	7
B	0	0	0	4	4
C	6	1	5	6	18
D	0	0	1	1	2

Table E.4: Development flow in Phase 2 (Project A)

AI Level	Coding (h)	Review (h)	Total (h)
None	0.19	0.85	1.04
Low	0.10	140.6*	140.7
Medium	42.7	817.1**	859.8

E.4.5 KPI 5 - Developer Survey

The survey includes 12 responses from developers in the pilot projects.

Training participation

The second survey received 12 responses. Seven participants reported no formal AI training, while two participants reported Silver-level training and three reported Gold-level training.

Tool usage

GitHub Copilot is used by all respondents. Cursor is used by some developers, and no additional tools are observed.

AI usage by task

AI is used mainly for coding, debugging, and understanding code. It is also used for testing and documentation. It is used less for design, architecture, and code review.

Table E.5: Development flow in Phase 2 (Project B)

AI Level	Coding (h)	Review (h)	Total (h)
High	74.0	757.0	831.0

Table E.6: Development flow in Phase 2 (Project C)

AI Level	Coding (h)	Review (h)	Total (h)
None	1.6	164.4	166.0
Low	0.13	292.6	292.7
Medium	195.8	2791.0	2986.8
High	67.5	174.7	242.2

Experience notes

Developers report improved flow in some cases, but AI outputs still require verification. Waiting time and interruptions appear, some developers report increased workload, and developers remain in control of decisions.

Open responses

Open-ended responses suggest that AI helps complete tasks faster and increase output within the same time. At the same time, AI can introduce fatigue and continuous work, waiting time interrupts focus, developers switch between tasks while waiting, and AI still struggles with complex systems. Developers also report that AI supports exploration and planning.

E.5 Summary of Phase 2 Detailed Results

The detailed Phase 2 results support the summary presented in Chapter 6. Coding is fast in many cases, review takes more time, verification effort remains present, and task complexity strongly affects results. Variation across projects increases. AI usage appears more often, but it does not remove the need for review and validation. Developer experience indicates both benefits and challenges.

Table E.7: Development flow in Phase 2 (Project D)

AI Level	Coding (h)	Review (h)	Total (h)
Medium	0.10	21.8	21.9
High	0.005	20.5	20.5

Table E.8: Verification in Phase 2 (Project A)

AI	Rework	Rounds
None	0%	2
Low	~50%	2
Medium	~33%	2

Table E.9: Verification in Phase 2 (Project B)

AI	Rework	Rounds
High	~75%	3

Table E.10: Verification in Phase 2 (Project C)

AI	Rework	Rounds
None	~20%	2
Low	~100%	5
Medium	~100%	10+
High	~75%	3

Table E.11: Verification in Phase 2 (Project D)

AI	Rework	Rounds
Medium	0%	1
High	0%	1

Table E.12: Defect signals in Phase 2

Project	Rate	Context
A	0%	-
B	0%	-
C	0%	-
D	0%	-